PriX: Private Accelerated Transformers via Secure Multi-Party Computation in JAX

Mehmet Ugurbil
mugurbil@gmail.com
https://github.com/mugurbil/prix

Abstract

Private inference of Transformer and Large Language Models (LLMs) via Secure Multi-Party Computation (MPC) faces a systems bottleneck: CrypTen-family frameworks execute secret-shared protocols over $\mathbb{Z}_{2^{64}}$ via eager, operator-by-operator dispatch in PyTorch. While recent algorithmic advances—Wavelet-encoded Look-Up Tables (Curl, Wave), Function Secret Sharing (SIGMA, Wave), piecewise low-degree polynomials (PUMA, BOLT, Nimbus), 1-round Fourier-series activations and ODE Softmax (SHAFT), and hybrid plaintext evaluators (Fission)—substantially reduce the cost of nonlinearities, their reference implementations are spread across separate codebases and pay kernel-launch and host-side dispatch overhead on every elementary ring operation.

We present **PriX**, an open-source JAX/XLA framework for private Transformer inference that implements seven MPC approximation paradigms (**CrypTen**, **PUMA/BOLT/Nimbus piecewise polynomials**, **SHAFT**, **Curl**, **SIGMA**, **Wave**, and **Fission**) under a single API while compiling local ring and FSS tree operations into fused XLA kernels. PriX combines (1) *exact limb-decomposed FP64 matrix multiplication* over $\mathbb{Z}_{2^{64}}$ that computes only the limb products surviving modulo 2^{64} (6 products with 22-bit limbs for contraction size $K \leq 512$, 10 products with 16-bit limbs otherwise); (2) *`@jax.jit` fusion* of the local steps of multi-round MPC and FSS circuits (carry-lookahead comparison rounds, DPF/DCF and Grotto prefix-parity trees, wavelet LUT gathers, Fourier-series combinations); and (3) *a pickle-free peer-to-peer TCP transport*, with optional mutual TLS, that exchanges batched openings as single messages and runs unchanged on one host or across machines. Across 10 MPC primitives and 5 Transformer blocks (`BertTiny`, `BertBase`, `GPT2`, `Llama`, `ModernBert`) in the five CrypTen-compatible modes, PriX has lower latency than the PyTorch CrypTen baseline in all 75 configurations on both a CPU and an NVIDIA Tesla T4 GPU, achieving Transformer-block speedups of **2.32×–7.77×** (geometric mean 4.14×) on CPU and **1.91×–3.08×** (geometric mean 2.33×) on GPU. Using PriX as a controlled testbed, we further conduct a cross-paradigm study of Function Secret Sharing (**SIGMA** and **Wave**) versus **SHAFT** and **Curl**, and analyze the accuracy and security trade-offs between deterministic and probabilistic truncation in wavelet LUT indexing versus ring multiplication.

1 Introduction

Deploying Transformer-based Large Language Models (LLMs) [28] on sensitive user prompts or proprietary model weights requires cryptographic privacy guarantees. Secure Multi-Party Computation (MPC) over the machine ring $\mathbb{Z}_{2^{64}}$ [20, 22, 15, 11] enables two or more distrusting parties to jointly evaluate neural networks on secret-shared inputs without revealing either the client's prompt or the server's parameters.

Over the past five years, CrypTen [15] has served as the foundational Python research platform for machine-learning-oriented MPC. Building upon CrypTen, a wave of recent cryptographic protocol innovations has dramatically improved the accuracy and round complexity of Transformer nonlinearities (GeLU, SiLU, Softmax, LayerNorm, RMSNorm):

- **Wavelet Approximations and Curl** [9, 23, 24]: Wavelet-based cryptographic function approximation—pioneered across Fully Homomorphic Encryption in **Ripple** [9], mixed-mode MPC in **Wave Hello to Privacy** [23], and private LLM evaluation in **Curl** [24]—compresses lookup tables via Haar and Biorthogonal-2.2 wavelet transforms paired with probabilistic truncation (TruncPr) [8], substantially reducing LUT communication and memory footprints while preserving high numerical fidelity.
- **SHAFT** [13] introduced 1-round trigonometric-triple Fourier-series evaluation for GeLU, SiLU, Erf, Sigmoid, and Tanh, alongside a constant-round Forward-Euler Ordinary Differential Equation (ODE) solver for Softmax.
- **Fission** [27] introduced a hybrid permute-and-reveal architecture that offloads elementwise nonlinearities to a non-colluding plaintext evaluator party.
- **PUMA** [7], **BOLT** [21], and **Nimbus** [18] replace GeLU, SiLU, and exp with piecewise low-degree polynomials guarded by one or two secure comparisons, avoiding lookup tables entirely: BOLT folds the input domain onto $|x|$ using the even symmetry of $\mathrm{GeLU}(x) - \mathrm{ReLU}(x)$, and Nimbus fits the pieces to the observed input distributions of Transformer activations.

Despite these algorithmic advances, MPC research for Transformers faces two systems obstacles:

1. **Ecosystem Fragmentation:** The original CrypTen repository was archived in May 2025, and Curl, SHAFT, and Fission live in separate CrypTen forks (PUMA lives in SecretFlow SPU), each of which needs compatibility patches to run on current Python, PyTorch, and JAX releases. Researchers conducting ablations across approximation paradigms must port models between distinct codebases.
2. **Eager Dispatch Overhead:** In eager PyTorch, every elementary bitwise shift, mask, and ring addition in an MPC circuit is a separate CPU or CUDA kernel launch with host-side Python control flow in between. A single 64-bit carry-lookahead comparison [16] executes 6 tree rounds of bitwise XOR and AND operations, so its local work is split into many tiny launches. In addition, because neither CPUs' BLAS libraries nor CUDA provide 64-bit integer (`int64`) GEMM, CrypTen's CUDA backend splits each operand into 3 (for contraction size $K < 256$) or 4 limbs and computes all 9 or 16 pairwise limb products in one batched FP64 `torch.matmul`, discarding the products that vanish modulo 2^{64} only afterwards; on CPU it falls back to PyTorch's unblocked `int64` matmul.

To address both challenges, we design and implement **PriX** (*Private Accelerated Transformers in JAX*), a modern, drop-in alternative to CrypTen built on the JAX/XLA compiler stack [5].

Contributions. Our contributions are:

1. **A Multi-Paradigm MPC and FSS Framework:** PriX implements **CrypTen** polynomial/Newton–Raphson approximations [15], **PUMA / BOLT / Nimbus**-style piecewise low-degree polynomials [7, 21, 18], **SHAFT** Fourier-series and ODE Softmax [13], **Curl** wavelet LUTs with TruncPr [24, 8], **SIGMA** Function Secret Sharing (DPF/DCF with

Grotto prefix trees) [10, 25], **Wave** mixed-mode FSS wavelet LUTs [23], and **Fission** hybrid evaluators [27] behind a single API on Python 3.10–3.13, tested on CPUs and CUDA GPUs. Researchers switch paradigms in one line (`mode="lut" | "wave" | "sigma" | "poly" | "shaft" | "crypten" | "fission"`) or mix methods per function and truncation scheme.

2. **XLA Kernel Fusion for MPC and FSS Circuits:** PriX compiles the local (non-communicating) steps of secret-share arithmetic, carry-lookahead comparison rounds, DPF/DCF and Grotto prefix-parity tree evaluations, wavelet LUT gathers, and Fourier-series combinations via `@jax.jit`, removing intermediate memory traffic and Python dispatch between elementary ring operations.

3. **Exact Limb-Decomposed $\mathbb{Z}_{2^{64}}$ Matrix Multiplication and Low-Overhead Collectives:** PriX computes only the limb products that survive modulo 2^{64} (6 instead of CrypTen's 9 for $K < 256$, 6 instead of 16 for $256 \le K \le 512$, and 10 instead of 16 for $K > 512$) inside one compiled kernel, and exchanges batched openings over a pickle-free peer-to-peer TCP transport with optional mutual TLS.

4. **CPU/GPU Evaluation and Cross-Paradigm Protocol Analysis:** We compare PriX against the PyTorch CrypTen baseline across 75 configurations on CPU and GPU (**2.32**×–**7.77**× Transformer-block speedups on CPU and **1.91**×–**3.08**× on GPU), and use PriX to conduct an apples-to-apples evaluation of **Wave**, **SIGMA**, **Curl**, and **SHAFT** alongside an analytical and empirical study of deterministic versus probabilistic truncation in LUT indexing and ring multiplication (Section 6).

2 Background and MPC Preliminaries

2.1 Fixed-Point Encoding and Secret Sharing over $\mathbb{Z}_{2^{64}}$

Real-valued tensors $x \in \mathbb{R}^{d_1 \times \cdots \times d_k}$ are encoded into the ring $\mathbb{Z}_{2^{64}}$ using a fixed-point scaling factor $S = 2^f$ (where $f = 16$ precision bits by default):

$$\mathsf{Encode}(x) = \lfloor x \cdot 2^f \rceil \bmod 2^{64}, \qquad \mathsf{Decode}(\hat{x}) = \frac{\mathsf{Signed}_{64}(\hat{x})}{2^f}. \tag{1}$$

Across N parties (typically $N = 2$), PriX maintains two secret-sharing representations:

- **Additive Arithmetic Sharing ($\langle x \rangle^A$):** Party $p \in \{0, \ldots, N-1\}$ holds $\langle x \rangle_p^A \in \mathbb{Z}_{2^{64}}$ such that $\sum_{p=0}^{N-1} \langle x \rangle_p^A \equiv \mathsf{Encode}(x) \pmod{2^{64}}$. Linear combinations $\langle x + y \rangle^A = \langle x \rangle^A + \langle y \rangle^A$ and public multiplications $\langle c \cdot x \rangle^A = c \cdot \langle x \rangle^A$ require zero communication (followed by local or interactive fixed-point truncation).

- **XOR Binary Sharing ($\langle x \rangle^B$):** Party p holds $\langle x \rangle_p^B \in \{0,1\}^{64}$ such that $\bigoplus_{p=0}^{N-1} \langle x \rangle_p^B = x$. Bitwise XOR ($\oplus$) and bit shifts ($\ll, \gg$) are local; bitwise AND ($\wedge$) requires 1 communication round via binary Beaver triples.

2.2 Multiplication, Truncation, and Nonlinearity Paradigms

Beaver Multiplication Triples. To compute the private product $\langle z \rangle^A = \langle x \cdot y \rangle^A$ (or matrix product $\langle XY \rangle^A$), the parties consume a pre-generated triple $(\langle a \rangle^A, \langle b \rangle^A, \langle c \rangle^A)$ with $c = a \cdot b \bmod$

2^{64} [2]. Parties locally compute $\langle \epsilon \rangle^A = \langle x \rangle^A - \langle a \rangle^A$ and $\langle \delta \rangle^A = \langle y \rangle^A - \langle b \rangle^A$, open (ϵ, δ) in a single communication round, and evaluate:

$$\langle x \cdot y \rangle_p^A = \langle c \rangle_p^A + \epsilon \langle b \rangle_p^A + \langle a \rangle_p^A \delta + \mathbb{I}[p = 0] \cdot (\epsilon\delta) \pmod{2^{64}}. \tag{2}$$

Because multiplying two 2^f-scaled integers yields scale 2^{2f}, the product must be truncated by f bits:

- *CrypTen Truncation (`crypten`):* With two parties, each party arithmetic-shifts its own share by f bits with no communication (local share truncation, as in CrypTen [15] and SecureML [20]); PriX folds this shift into the compiled Beaver combination. The result is correct up to one unit in the last place, except with probability about $|x|/2^{63}$ for a ring value x, when the shares wrap and the result is wrong by a large amount. With more than two parties, CrypTen's masked wrap-count protocol corrects for the wraps θ_x in one extra round.
- *Probabilistic Truncation (`egk`):* Following Escudero et al. [8] and Curl [24], parties mask $\langle z \rangle^A$ with a correlated random pair $(\langle r \rangle^A, \langle r \gg f \rangle^A)$, reveal $z + r$, and subtract $\langle r \gg f \rangle^A$ from $(z + r) \gg f$.

Seven Nonlinear Approximation Backends. Table 1 summarizes the seven approximation modes unified in PriX:

1. **CrypTen Mode (`mode="crypten"`)** [15]: Approximates $\exp(x) \approx (1 + x/2^m)^{2^m}$ via $m = 8$ repeated squarings, $1/x$ and $1/\sqrt{x}$ via Newton–Raphson iterations, and GeLU(x) via Taylor-series $\mathrm{erf}(x/\sqrt{2})$.
2. **PUMA / BOLT / Nimbus Piecewise Polynomial Mode (`mode="poly"`)** [7, 21, 18]: Evaluates GeLU, SiLU, Tanh, Sigmoid, and Erf with one batched secure comparison (computing $[\langle x \rangle^A < 0]$ and $[|\langle x \rangle^A| < w]$ in a single carry-lookahead circuit) followed by a degree-4 polynomial on $|x|$ (using BOLT's symmetry folding $\mathrm{ReLU}(x) + \mathbb{I}[|x| < w]\, P(|x|)$ for GeLU and SiLU), and exp with a two-piece degree-4 polynomial on $(-5, 0]$ and $[0, 2.5]$ whose pieces share one power basis. No lookup tables are used. The coefficients are least-squares fits over the stated intervals rather than the distribution-aware fits of Nimbus.
3. **SHAFT Mode (`mode="shaft"`)** [13]: Evaluates GeLU, SiLU, Erf, Sigmoid, and Tanh via a Fourier sine series $\sum_{k=1}^{K} \beta_k \sin(k\omega x)$ whose evaluation takes *1 communication round*: parties reveal $\delta = x - t$ against a pre-shared trigonometric triple $(\langle t \rangle^A, \langle \sin(k\omega t) \rangle^A, \langle \cos(k\omega t) \rangle^A)$ and apply the angle-addition identity locally:

   $$\langle \sin(k\omega x) \rangle^A = \sin(k\omega\delta) \langle \cos(k\omega t) \rangle^A + \cos(k\omega\delta) \langle \sin(k\omega t) \rangle^A. \tag{3}$$

   The series is only accurate within one period, so for GeLU and SiLU PriX first computes $[x < 0]$ and $[|x| < w]$ in one batched comparison, evaluates the series on $|x|$, and selects between it and $\mathrm{ReLU}(x)$; the comparison dominates the round count (11 rounds per GeLU in our measurements). For Softmax, SHAFT integrates the differential equation $y'(t) = y(t) \odot (x - \langle y(t), x \rangle)$ over $t \in [0, 1]$ using a fixed number of Forward-Euler steps, without any division, reciprocal, or secure max.
4. **Curl Wavelet LUT Mode (`mode="lut"`)** [24]: Builds on wavelet-based function approximation [9, 23]: lookup tables are compressed with Haar and Biorthogonal-2.2 wavelet transforms into coarse coefficient tables, secret-shared lookups are evaluated with one-hot correlated randomness [24], and Biorthogonal-2.2 tables are interpolated piecewise-linearly after probabilistic truncation TruncPr [8].

Table 1: Summary of the seven MPC approximation paradigms implemented in PriX (default configuration of each mode).

Mode	Truncation	GeLU / SiLU	Softmax	Reciprocal / InvSqrt
`"crypten"` [15]	Local shift	Taylor erf / Sigmoid	Secure max + limit exp + NR recip.	Newton–Raphson
`"poly"` [7, 21, 18]	Local shift	Piecewise degree-4 poly	Secure max + piecewise exp + NR recip.	Newton–Raphson
`"shaft"` [13]	Local shift	Fourier series + comparison	Forward-Euler ODE	Newton–Raphson
`"lut"` [24]	TruncPr [8]	Bior-2.2 LUT (one-hot)	Secure max + Bior-2.2 exp / recip. LUTs	Bior-2.2 LUT (one-hot)
`"sigma"` [10]	DPF/DCF + TruncPr	Haar LUT (Grotto DPF)	DPF max + Haar exp / recip. LUTs	Haar LUT (Grotto DPF)
`"wave"` [23]	DPF/DCF (`wave`) + TruncPr	bior(5, 3) LUT (DPF+DCF)	DPF max + bior(5, 3) exp / recip. LUTs	bior(5, 3) LUT (DPF+DCF)
`"fission"` [27]	Deferred / Evaluator	Permute & reveal	Permute & reveal	Permute & reveal

5. **SIGMA Function Secret Sharing Mode (`mode="sigma"`)** [10]: Uses 2-party Distributed Point Functions (DPFs), Distributed Comparison Functions (DCFs) [3, 4], and Grotto prefix-parity trees [25] to evaluate secure comparisons and 0th-order Haar LUTs in 2 online rounds.

6. **Wave Mixed-Mode FSS Wavelet Mode (`mode="wave"`)** [23]: Combines $(J + \mathrm{j})$-level Grotto DPF segment parities and j-level arithmetic DCFs (or 0-round probabilistic share splitting `"wave_prob"`) with 1st-order Biorthogonal bior$(5, 3)$ and Haar bior$(1, 1)$ wavelet tables and generalized Beaver tuples in 2 online rounds (Section 6).

7. **Fission Evaluator Mode (`mode="fission"`)** [27]: Masks and randomly permutes shares before revealing them to a non-colluding plaintext evaluator party, which computes exact float32 nonlinearities and returns fresh secret shares. The evaluator learns the permuted plaintext values, so the security argument relies on the evaluator not colluding with the MPC parties and on partitioning values across evaluators [27].

2.3 Threat Model

PriX targets the setting of CrypTen and its descendants: $N \geq 2$ computing parties, typically two, that hold additive shares of the model weights and inputs and are *semi-honest*, i.e. they follow the protocol but may try to learn from what they see. We assume no collusion between computing parties. Correlated randomness comes from a dealer: either Party 0 itself (`TFP`, which is only meaningful for benchmarking, since Party 0 then knows every mask) or a separate non-colluding dealer process (`TTP`). In `fission` mode, an additional evaluator party sees masked, permuted plaintext values of each nonlinearity's input and must not collude with the computing parties (permutations are derived from a seed shared only among the computing parties, so neither the evaluator nor the `TTP` dealer learns them). Malicious parties, active attacks, and side channels such as timing are out of scope, as are model-extraction attacks on the revealed output. Section 7 lists the ways in which the current implementation falls short of even this model.

3 PriX Architecture and XLA Compiler Optimizations

Figure 1 presents the four-layer architecture of PriX. Eager PyTorch frameworks launch each ring-arithmetic or bitwise operation as its own kernel. PriX instead expresses the local (non-communicating) phase between two openings of a protocol as a pure function over 64-bit JAX arrays (`jnp.int64` / `jnp.uint64`) and compiles it with `@jax.jit`, so each such phase becomes one XLA executable. Communication remains on the host between compiled phases.

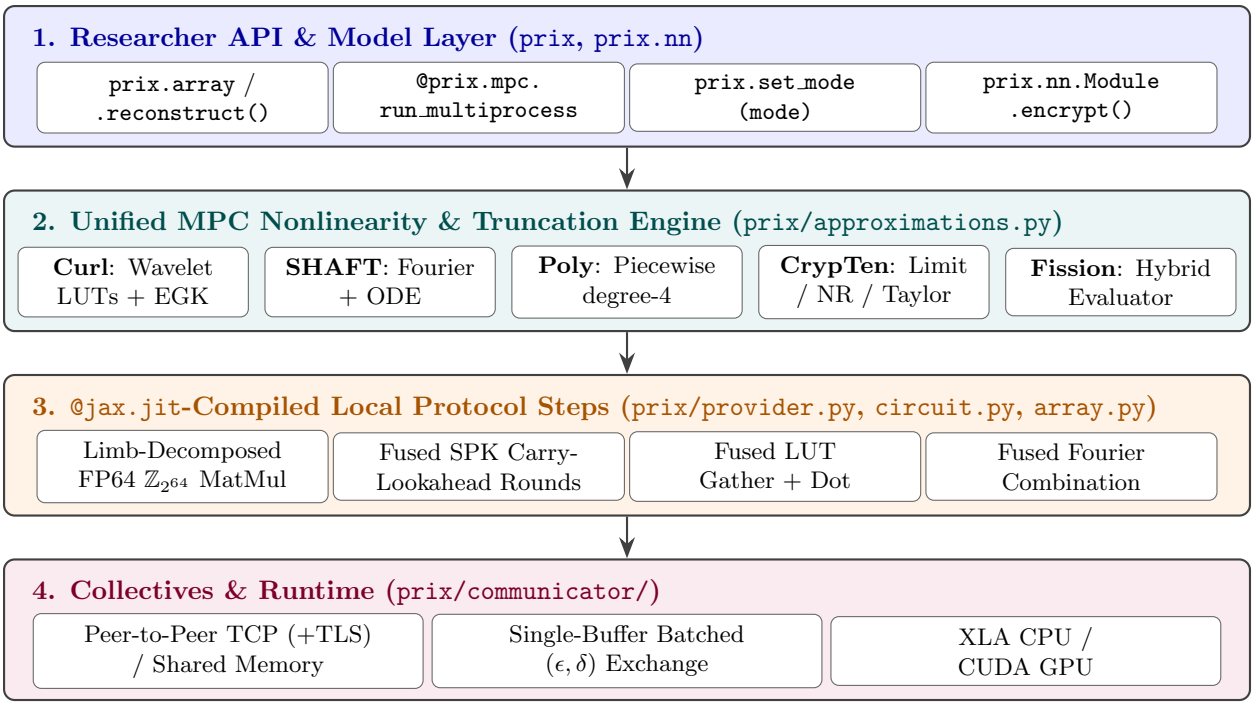


Figure 1: System architecture of **PriX**, from the researcher-facing Python API down to compiled local protocol steps and transports.

3.1 Exact Limb-Decomposed FP64 Matrix Multiplication over $\mathbb{Z}_{2^{64}}$

Neither CPU BLAS libraries nor cuBLAS provide 64-bit integer (`int64`) GEMM, so PyTorch's `int64` matmul runs on generic unblocked loops. Double-precision GEMM, by contrast, is available in optimized form on both (AVX/FMA on CPU, `cublasDgemm` on GPU). Note that on the Tesla T4 used in our GPU experiments, FP64 throughput is a small fraction of FP32 throughput and does not use tensor cores.

To compute $C = AB \bmod 2^{64}$ for $A \in \mathbb{Z}_{2^{64}}^{M\times K}$ and $B \in \mathbb{Z}_{2^{64}}^{K\times N}$ exactly in FP64, we split each unsigned 64-bit entry into L limbs of w bits, $A = \sum_i A_i 2^{wi}$ and $B = \sum_j B_j 2^{wj}$. Because only the result modulo 2^{64} is needed, every limb pair with $(i+j)\,w \geq 64$ contributes a multiple of 2^{64} and can be skipped:

$$AB \equiv \sum_{(i+j)\,w<64} (A_i B_j)\, 2^{w(i+j)} \pmod{2^{64}}. \tag{4}$$

PriX uses two limb configurations, chosen by the contraction size K:

- $K \leq 512$: $L = 3$ limbs of $w = 22$ bits (A_2, B_2 have 20 bits), so only the **6** pairs with $i + j \leq 2$ are computed. Each product entry is below $K \cdot 2^{44} \leq 2^{53}$, so every FP64 GEMM is exact.
- $K > 512$: $L = 4$ limbs of $w = 16$ bits, so only the **10** pairs with $i + j \leq 3$ are computed. On GPU each pair is a separate slice of one batched GEMM. Each pair product is converted to `uint64` individually, which is exact for $K \cdot 2^{32} \leq 2^{53}$, i.e. $K \leq 2^{21}$. On CPU, each pair product is likewise converted to `uint64` before products of equal shift are added, so both backends share the same $K \leq 2^{21}$ bound.
- $K > 2^{21}$: the contraction is split into chunks of at most 2^{21}. Each chunk is multiplied exactly as above, and the `uint64` partial results are added; wrap-around addition is exact modulo

2^{64}, so the result is bit-exact for any K. The chunking is decided at trace time, so it adds no overhead when $K \leq 2^{21}$.

CrypTen's CUDA backend [15] uses 3 limbs for $K < 256$ and 4 limbs otherwise, and computes *all* 9 or 16 limb products in one batched `torch.matmul` before discarding the vanishing ones. PriX therefore performs 6 instead of 9 GEMMs for $K < 256$, 6 instead of 16 for $256 \leq K \leq 512$, and 10 instead of 16 for $K > 512$. On GPU, PriX stacks the surviving limb pairs into one batched GEMM, and the limb extraction, GEMM, conversion, shifts, and modular accumulation are compiled together with the Beaver combination $\langle C\rangle^A + \epsilon\langle B\rangle^A + \langle A\rangle^A\delta + \mathbb{I}[p = 0](\epsilon\delta)$. On CPU, CrypTen uses PyTorch's `int64` matmul directly, so PriX's advantage there comes mainly from using FP64 BLAS.

3.2 Compiled Set-Propagate-Kill Comparison Circuits

Private comparisons ($\langle x\rangle^A < 0$, ReLU, Max) extract the most significant bit of a secret-shared ring element by converting $\langle x\rangle^A$ into binary shares with a parallel-prefix carry-lookahead circuit. Like CrypTen, PriX uses a carry-lookahead adder built from set/propagate (Set-Propagate-Kill, SPK) carry signals [6], arranged as a Kogge–Stone parallel-prefix tree [16] with $\log_2 64 = 6$ rounds. In round r with stride $s = 2^r$, the set and propagate bit-vectors are updated as

$$S^{(r+1)} = S^{(r)} \oplus \big(P^{(r)} \wedge (S^{(r)} \ll s)\big), \qquad P^{(r+1)} = P^{(r)} \wedge (P^{(r)} \ll s), \tag{5}$$

where XOR can replace OR because the two terms are never both set. As in CrypTen, both AND gates of a round are opened in one batched binary-Beaver exchange. PriX differs in how the local work is executed:

- The dealer generates the binary triples for all 6 rounds in one call (`generate_spk_triples`) instead of one call per round.
- The local work between two consecutive openings—applying the previous round's Beaver correction, masking, shifting, and preparing the next round's masked operands—is one compiled kernel (`_spk_step_fused` in `circuit.py`). The first and last rounds are fused with the arithmetic-to-binary conversion and with the binary-to-arithmetic conversion of the sign bit, respectively (`_a2b_post_and_spk_round0_pre`, `_spk_round_last_and_ltz_b2a_pre`, `_b2a_combine`).

3.3 Compiled Wavelet LUT Evaluation and Fourier Combination

In Curl mode (`mode="lut"`), a nonlinearity such as InvSqrt, Exp, GeLU, or SiLU truncates the secret input to a table index, reveals the index masked by a one-hot correlation, rotates the one-hot share accordingly, and takes its dot product with the (Biorthogonal-2.2) coefficient tables before interpolating. In PriX:

- Each wavelet table is built on first use, placed on the accelerator, and cached for the rest of the session (`LookupTables(device)`), so table construction is excluded from the timed runs by the warmup.
- The rotation and dot product against one table, or against the two Biorthogonal-2.2 tables used for interpolation, are one compiled kernel (`_lut_gather_dot`, `_bior_lut_gather_dot` in `array.py`).

- In SHAFT mode, after the single opening of $\delta = x - t$, the evaluation of Equation 3 for all K terms—computing $\sin(k\omega\delta)$ and $\cos(k\omega\delta)$, scaling them by β_k, multiplying against the triple shares, and summing—is one compiled kernel (`_fourier_linear_comb_jit` in `approximations.py`).

3.4 Batched Openings and Transports

Every private multiplication opens two masked tensors, $\epsilon = x - a$ and $\delta = y - b$. As in CrypTen, both are opened in one round. When ϵ and δ have different shapes (e.g. $\epsilon \in \mathbb{Z}_{2^{64}}^{M\times K}$ and $\delta \in \mathbb{Z}_{2^{64}}^{K\times N}$ in a matrix multiplication), PriX passes both to one batched `all_reduce`, which sends them as a single message and splits the result.

PriX's default transport is **peer-to-peer TCP**: every pair of processes (MPC parties, the TTP dealer if used, and Fission evaluators) shares one direct connection, with no coordinator in the data path, and the same code runs on one host over loopback or across machines. Messages use a small binary format that carries only plain data and numeric arrays, so, unlike pickle, decoding a malicious peer's message cannot execute code. Sockets are non-blocking, and every wait keeps flushing queued outgoing data, so two parties that send each other large openings at the same time cannot deadlock. Connections can optionally use mutual TLS with a shared certificate authority; all experiments here run without TLS, like CrypTen's transport. For single-host simulation, PriX also offers a **shared-memory transport**: a 4-slot ring buffer (`multiprocessing.RawArray`) per party with a 1-byte pipe message as the readiness signal, with slots of 2^{21} `int64` values (16 MiB) and a pickled-queue fallback for larger openings. CrypTen uses `torch.distributed` with the `gloo` backend over loopback TCP. Section 5.4 compares all three.

4 Researcher Ergonomics and Unified API

A central design goal of PriX is enabling cryptography and machine learning researchers to prototype and benchmark private Transformer protocols with minimal boilerplate. Listing 1 shows a multi-party ablation that evaluates a pre-norm Transformer block under all five approximation modes.

Listing 1: Evaluating a private Transformer block under all five MPC approximation modes in PriX.

```
import numpy as np
import prix
import prix.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, d_model=64, n_heads=4, d_ff=256):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(
            d_ff, d_model))

    def forward(self, x):
        h = self.ln1(x)
        x = x + self.attn(h, h, h)
        return x + self.ffn(self.ln2(x))
```

```python
def evaluate_private_block(tokens):
    # Party 0 secret-shares the model weights; party 1 secret-shares the token
        embeddings
    model = TransformerBlock().encrypt(src=0)
    x = prix.array(tokens, src=1)
    return model(x).reconstruct()

tokens = np.random.randn(1, 16, 64).astype(np.float32)
for mode in ["crypten", "poly", "shaft", "lut", "fission"]:
    out = prix.run(evaluate_private_block, tokens, world_size=2, mode=mode)
```

5 Experimental Evaluation

5.1 Experimental Setup

We compare **PriX (JAX)** against a PyTorch **CrypTen** baseline [15]: the CrypTen fork that serves as the reference implementation of Curl [24] and Fission [27]. SHAFT's Fourier activations and ODE Softmax [13], and the piecewise polynomials of `poly` mode, are added to this baseline by our benchmark script. The SHAFT functions follow the SHAFT authors' CrypTen-based reference implementation, which computes GeLU/SiLU with ReLU followed by a separate range comparison. PUMA, BOLT, and Nimbus have no CrypTen implementation, so the `poly` functions are ports of PriX's algorithms with the same coefficients, intervals, batched comparisons, and power-basis sharing. Where PriX seeds Newton–Raphson iterations with the limit approximation of exp regardless of the configured exp method, the baseline is configured the same way. Both frameworks run 2-party semi-honest protocols with the Trusted First Party (`TFP`) dealer, $f = 16$ fractional bits, identical inputs, and identical configurations: each mode's YAML is merged over the framework's default configuration, and the CrypTen side then takes PriX's value for every setting both define, including the adjusted LUT ranges of Section 5.5 (a configuration change only; the baseline code is unchanged). The five modes are:

1. **CrypTen approximations** (`crypten`) [15];
2. **Piecewise polynomials** (`poly`) [7, 21, 18];
3. **SHAFT Fourier + ODE** (`shaft`) [13];
4. **Curl wavelet LUTs with** TruncPr (`lut`) [24]; and
5. **Fission hybrid evaluator** (`fission`) [27], with one evaluator process.

We benchmark 10 MPC primitives and nonlinearities (MatMul 128×128, Mul 128×128, ReLU 32×64, Exp 32×64, Reciprocal 32×64, InvSqrt 32×64, GeLU 32×64, SiLU 32×64, Softmax 4×32×32, LayerNorm $4 \times 32 \times 64$) and 5 Transformer configurations with sequence length $L = 16$: `BertTiny` (2 blocks, $d = 128$, $H = 2$), `BertBase` (1 block, $d = 768$, $H = 12$), `GPT2` (1 block, $d = 768$, $H = 12$), `Llama` (1 block, $d = 768$, $H = 12$, $H_{\text{kv}} = 4$, RoPE + SwiGLU + RMSNorm, grouped-query attention), and `ModernBert` (1 block, $d = 768$, $H = 12$, RoPE + GeGLU). Weights and inputs are random; we measure latency only, not model accuracy.

Methodology. All parties run as separate processes on one host and communicate over loopback TCP: CrypTen through `torch.distributed/gloo`, and PriX through its own peer-to-peer TCP transport (Section 3.4), without TLS in either case. No network latency or bandwidth limit is

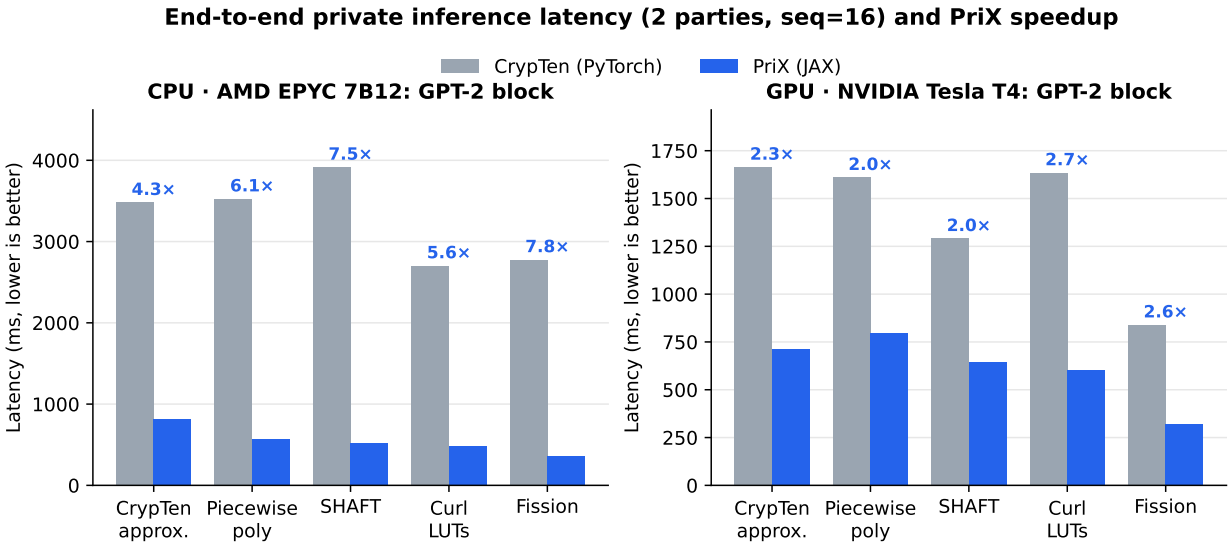


Figure 2: End-to-end latency of one GPT-2 block ($d = 768$, $L = 16$, 2 parties) for CrypTen and PriX under each approximation mode, on CPU (left) and GPU (right). Labels give the PriX speedup.

Table 2: **CPU Transformer-block latency (ms) and PriX speedup** for all five approximation modes ($N = 2$ parties, $L = 16$; median of 3 timed runs). Host: AMD EPYC 7B12, 24 vCPUs; Python 3.13, JAX/jaxlib 0.12.0, PyTorch 2.14.1 (CPU). PriX uses its default peer-to-peer TCP transport; CrypTen uses `gloo` over TCP. All entries were measured in one session. Lower is better.

	CrypTen Approx.		Piecewise Poly		SHAFT (Fourier+ODE)		Curl Wavelet LUTs		Fission Evaluator	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1423.31	**582.27 (2.44×)**	1687.14	**560.17 (3.01×)**	1010.66	**434.93 (2.32×)**	1106.85	**446.87 (2.48×)**	443.46	**163.33 (2.72×)**
BertBase (1-Blk)	2950.74	**737.85 (4.00×)**	2968.03	**733.66 (4.05×)**	3154.52	**735.38 (4.29×)**	2167.78	**582.69 (3.72×)**	2081.62	**449.65 (4.63×)**
GPT2 (1-Blk)	3480.96	**809.60 (4.30×)**	3524.84	**574.84 (6.13×)**	3915.11	**519.49 (7.54×)**	2702.19	**482.07 (5.61×)**	2776.97	**357.22 (7.77×)**
Llama (1-Blk)	2317.26	**566.54 (4.09×)**	2779.46	**612.08 (4.54×)**	3021.76	**600.14 (5.04×)**	2068.08	**525.26 (3.94×)**	2244.43	**463.35 (4.84×)**
ModernBert (1-Blk)	1899.87	**557.62 (3.41×)**	2099.36	**514.43 (4.08×)**	2236.08	**450.99 (4.96×)**	1685.12	**425.10 (3.96×)**	1432.81	**307.39 (4.66×)**

emulated. Each primitive is run 2 times untimed (to exclude JIT compilation and cache warmup) and then timed 5 times; each Transformer block is warmed up once and timed 3 times. Parties synchronize with a barrier before every timed run, device work is synchronized before the timer stops, and we report the median. Communication statistics (rounds and bytes) were recorded in all runs. On each platform, all configurations (CrypTen, and PriX over each of its three transports; Section 5.4) were measured in one session (on CPU, rotating the order of the four runs between modes so that slow drift of the host does not favor one of them). CPU experiments use the host listed in Table 2's caption. GPU experiments use a Google Colab instance with one NVIDIA Tesla T4. With so few repetitions, differences of roughly 10% or less between the two frameworks should be treated as within run-to-run noise.

Figures 2–4 summarize the results; Tables 2–5 give the underlying numbers.

5.2 CPU Benchmarks

Tables 2 and 3 report the CPU results. PriX has lower latency than CrypTen in all 75 CPU configurations: Transformer-block speedups range from **2.32**× to **7.77**× (geometric mean 4.14×), and primitive speedups from 1.06× (Mul, `shaft` mode) to 13.25× (InvSqrt, `fission` mode).

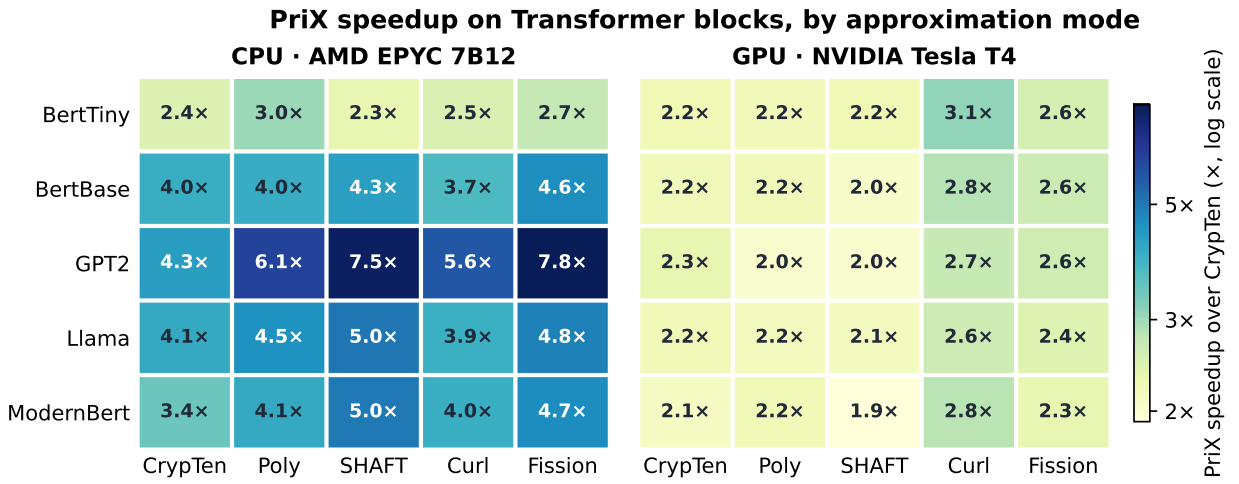


Figure 3: PriX speedup over CrypTen on all five Transformer blocks and all five approximation modes, on CPU and GPU (log color scale). The CrypTen `fission` column on CPU is affected by contention from the evaluator process (Section 5).

Table 3: **CPU primitive and nonlinearity latency (ms, CrypTen / PriX)** and PriX speedup (median of 5 timed runs; same host and session as Table 2).

Operation (Shape)	CrypTen Approx.	Piecewise Poly	SHAFT (Fourier+ODE)	Curl Wavelet LUTs	Fission Evaluator
Private MatMul (128 × 128)	23.89 / **5.82** (**4.10**×)	24.54 / **6.55** (**3.75**×)	25.05 / **6.15** (**4.08**×)	28.34 / **7.49** (**3.78**×)	29.61 / **7.70** (**3.84**×)
Private Mul (128 × 128)	5.29 / **4.95** (**1.07**×)	5.89 / **3.65** (**1.61**×)	4.85 / **4.57** (**1.06**×)	9.48 / **6.39** (**1.48**×)	6.90 / **4.47** (**1.54**×)
Comparison / ReLU (32 × 64)	32.52 / **9.46** (**3.44**×)	36.94 / **9.50** (**3.89**×)	31.10 / **10.06** (**3.09**×)	30.12 / **13.09** (**2.30**×)	41.98 / **13.25** (**3.17**×)
Exp (32 × 64)	21.08 / **14.97** (**1.41**×)	52.61 / **19.06** (**2.76**×)	20.77 / **17.73** (**1.17**×)	68.19 / **14.94** (**4.56**×)	10.90 / **2.67** (**4.08**×)
Reciprocal (32 × 64)	105.89 / **49.65** (**2.13**×)	128.86 / **40.38** (**3.19**×)	95.99 / **59.46** (**1.61**×)	50.35 / **9.53** (**5.29**×)	19.56 / **2.67** (**7.33**×)
InvSqrt (32 × 64)	65.45 / **34.43** (**1.90**×)	67.89 / **31.04** (**2.19**×)	69.75 / **45.42** (**1.54**×)	122.58 / **19.72** (**6.22**×)	20.26 / **1.53** (**13.25**×)
GeLU (32 × 64)	114.98 / **62.66** (**1.83**×)	59.24 / **24.05** (**2.46**×)	83.85 / **24.27** (**3.45**×)	34.11 / **8.18** (**4.17**×)	11.13 / **2.60** (**4.29**×)
SiLU (32 × 64)	83.53 / **39.35** (**2.12**×)	53.84 / **23.47** (**2.29**×)	74.87 / **34.21** (**2.19**×)	35.45 / **7.40** (**4.79**×)	12.00 / **1.80** (**6.68**×)
Softmax (4 × 32 × 32)	382.48 / **169.38** (**2.26**×)	442.88 / **159.95** (**2.77**×)	178.78 / **136.92** (**1.31**×)	378.89 / **120.87** (**3.13**×)	18.56 / **2.87** (**6.46**×)
LayerNorm (4 × 32 × 64)	75.40 / **28.37** (**2.66**×)	72.25 / **31.84** (**2.27**×)	76.07 / **40.36** (**1.88**×)	55.58 / **13.91** (**4.00**×)	21.72 / **3.07** (**7.07**×)

- **Run-to-run noise.** ReLU has the same protocol in every mode, so its spread across modes gives a rough noise estimate. Within this session, CrypTen's ReLU took 30.1–36.9 ms in four modes and 42.0 ms in `fission` mode, where an extra evaluator process competes for the same host; PriX's took 9.5–13.3 ms. Single-operation speedups below about 1.2× (Mul in `crypten` and `shaft` modes, Exp in `shaft` mode) are within this noise. The host is a shared cloud VM, and absolute latencies vary between sessions: both frameworks ran about 25–40% slower (median) in this session than in an earlier one with the same software. Only comparisons within one session are meaningful, which is why all CPU numbers in this paper come from a single session.

- **Matrix multiplication.** On CPU, CrypTen uses PyTorch's `int64` matmul, which has no BLAS backend. PriX uses FP64 GEMM on limbs (Section 3.1), which explains the 3.8–4.1× MatMul speedups in every mode. Linear layers dominate the $d = 768$ blocks, which is consistent with their larger end-to-end speedups (3.4–7.8×) compared with `BertTiny` (2.3–3.0×).

- **Comparisons and LUTs.** ReLU is 2.3–3.9× faster, consistent with fusing the local work of each SPK round and with its 20% smaller openings (Section 5.4). In `lut` mode, InvSqrt drops from 122.6 to 19.7 ms (6.22×), SiLU from 35.5 to 7.4 ms (4.79×), and LayerNorm from

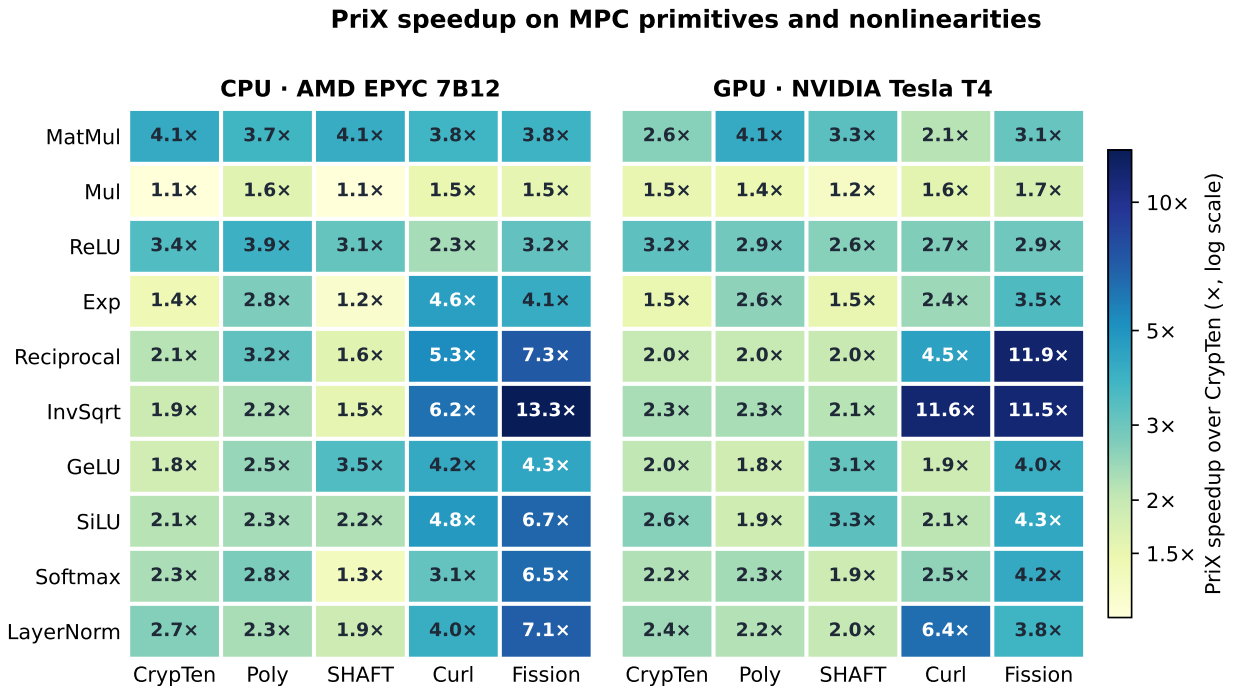


Figure 4: PriX speedup over CrypTen on the 10 MPC primitives and nonlinearities, on CPU and GPU (log color scale).

Table 4: **GPU (NVIDIA Tesla T4, Google Colab) Transformer-block latency (ms) and PriX speedup** for all five approximation modes ($N = 2$ parties, $L = 16$; median of 3 timed runs). PriX uses its default peer-to-peer TCP transport; CrypTen uses `gloo` over TCP. All entries were measured in one session. Lower is better.

	CrypTen Approx.		Piecewise Poly		SHAFT (Fourier+ODE)		Curl Wavelet LUTs		Fission Evaluator	
Model	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**	CrypTen	**PriX (Speedup)**
BertTiny (2-Blk)	1935.00	**867.19 (2.23×)**	1815.61	**826.13 (2.20×)**	1197.88	**537.35 (2.23×)**	1968.48	**639.19 (3.08×)**	396.46	**155.38 (2.55×)**
BertBase (1-Blk)	1662.57	**761.04 (2.18×)**	1676.78	**763.90 (2.20×)**	1321.44	**650.66 (2.03×)**	1694.65	**600.76 (2.82×)**	926.40	**350.81 (2.64×)**
GPT2 (1-Blk)	1663.77	**712.28 (2.34×)**	1609.94	**794.24 (2.03×)**	1290.56	**645.33 (2.00×)**	1634.65	**604.11 (2.71×)**	838.81	**320.92 (2.61×)**
Llama (1-Blk)	1632.13	**739.13 (2.21×)**	1580.11	**729.44 (2.17×)**	1286.35	**603.96 (2.13×)**	1728.45	**658.25 (2.63×)**	983.09	**402.74 (2.44×)**
ModernBert (1-Blk)	1445.26	**686.65 (2.10×)**	1390.71	**628.84 (2.21×)**	1085.58	**568.86 (1.91×)**	1480.95	**531.07 (2.79×)**	689.95	**297.91 (2.32×)**

55.6 to 13.9 ms (4.00×).

- **Piecewise polynomials.** `poly` mode makes GeLU and SiLU cheaper than `crypten` mode in both frameworks (in PriX, 24.1 vs. 62.7 ms and 23.5 vs. 39.4 ms), but more expensive than Curl's LUTs (8.2 and 7.4 ms). For exp, the piecewise polynomial is *slower* than the limit approximation (19.1 vs. 15.0 ms in PriX) because its secure comparisons cost more than 8 squarings.

5.3 GPU Benchmarks (NVIDIA Tesla T4, CUDA)

Tables 4 and 5 report the GPU results. PriX has lower latency in all 75 GPU configurations, with Transformer-block speedups of **1.91×–3.08×** (geometric mean 2.33×) and primitive speedups from 1.23× (Mul, `shaft` mode) to 11.87× (Reciprocal, `fission` mode).

- **Run-to-run noise.** CrypTen's ReLU took 40.8–50.4 ms across the five modes; PriX's took

Table 5: **GPU (NVIDIA Tesla T4) primitive and nonlinearity latency (ms, CrypTen / PriX)** and PriX speedup (median of 5 timed runs; same session as Table 4).

Operation (Shape)	CrypTen Approx.	Piecewise Poly	SHAFT (Fourier+ODE)	Curl Wavelet LUTs	Fission Evaluator
Private MatMul (128 × 128)	11.45 / **4.35 (2.63×)**	12.66 / **3.07 (4.12×)**	11.87 / **3.61 (3.29×)**	16.47 / **7.71 (2.14×)**	11.13 / **3.64 (3.05×)**
Private Mul (128 × 128)	4.91 / **3.23 (1.52×)**	4.51 / **3.12 (1.45×)**	4.08 / **3.31 (1.23×)**	8.59 / **5.42 (1.59×)**	4.99 / **2.89 (1.73×)**
Comparison / ReLU (32 × 64)	50.43 / **15.73 (3.21×)**	44.18 / **15.18 (2.91×)**	41.51 / **15.77 (2.63×)**	40.80 / **14.97 (2.73×)**	45.86 / **15.65 (2.93×)**
Exp (32 × 64)	28.30 / **19.07 (1.48×)**	79.72 / **31.14 (2.56×)**	27.03 / **17.71 (1.53×)**	67.40 / **28.30 (2.38×)**	6.85 / **1.97 (3.49×)**
Reciprocal (32 × 64)	129.92 / **65.24 (1.99×)**	134.66 / **68.01 (1.98×)**	137.68 / **69.79 (1.97×)**	56.21 / **12.59 (4.47×)**	17.61 / **1.48 (11.87×)**
InvSqrt (32 × 64)	90.38 / **39.73 (2.28×)**	87.97 / **38.64 (2.28×)**	85.92 / **40.36 (2.13×)**	141.19 / **12.20 (11.57×)**	17.45 / **1.52 (11.49×)**
GeLU (32 × 64)	150.42 / **73.75 (2.04×)**	68.45 / **37.10 (1.84×)**	92.71 / **29.91 (3.10×)**	23.39 / **12.35 (1.89×)**	6.81 / **1.71 (3.98×)**
SiLU (32 × 64)	132.88 / **50.49 (2.63×)**	67.45 / **36.06 (1.87×)**	90.73 / **27.87 (3.26×)**	26.20 / **12.55 (2.09×)**	7.30 / **1.71 (4.26×)**
Softmax (4 × 32 × 32)	551.32 / **251.27 (2.19×)**	586.34 / **252.38 (2.32×)**	199.67 / **106.15 (1.88×)**	507.15 / **200.35 (2.53×)**	7.23 / **1.74 (4.15×)**
LayerNorm (4 × 32 × 64)	103.48 / **43.67 (2.37×)**	103.03 / **47.66 (2.16×)**	99.32 / **49.61 (2.00×)**	158.90 / **24.92 (6.38×)**	9.46 / **2.48 (3.81×)**

15.0–15.8 ms. PriX's MatMul was slower in `lut` mode (7.7 ms vs. 3.1–4.4 ms), where products use TruncPr rather than the communication-free local shift.

- **Comparisons.** CrypTen's ReLU on the GPU (41–50 ms) is slower than on CPU, consistent with many small kernel launches per SPK round. PriX takes 15.0–15.8 ms, a 2.6–3.2× speedup.
- **Wavelet LUTs.** In `lut` mode, InvSqrt drops from 141.2 to 12.2 ms (11.57×) and LayerNorm from 158.9 to 24.9 ms (6.38×). Both frameworks use the same Biorthogonal-2.2 configuration (one interval with a pair of slope/intercept tables). The gain comes from executing the same protocol with fewer launches and a faster local transport, not from a different algorithm.
- **SHAFT and piecewise polynomials.** Fourier GeLU drops from 92.7 to 29.9 ms (3.10×) and ODE Softmax from 199.7 to 106.2 ms (1.88×). Piecewise-polynomial GeLU and SiLU take 36–37 ms in PriX, between SHAFT (28–30 ms) and the `crypten` approximations (50–74 ms). As on CPU, the piecewise exp is slower than the limit approximation (31.1 vs. 19.1 ms).
- **End-to-end.** Transformer-block speedups are smaller on GPU (1.91–3.08×) than on CPU. Plausible causes are the T4's low FP64 throughput and host-side communication, which faster GPU kernels do not reduce. We have not profiled this split.

5.4 Transport Comparison: How Much Does the Transport Matter?

CrypTen and PriX use different transports (Section 3.4), so the speedups above combine PriX's compiled local computation with differences in how messages are moved. To separate the two, we ran PriX over each of its three transports in the same sessions as Tables 2–5: its default peer-to-peer TCP transport (the configuration in those tables); `torch.distributed/gloo` over TCP, i.e. CrypTen's own transport, by routing PriX's two-party `all_reduce` through it; and the shared-memory transport. Everything else in PriX (compiled kernels, limb matrix multiplication, batching, host staging) is unchanged, and all three variants produce the same results. Table 6 and Figure 5 summarize the results as geometric means.

- **Most of the speedup does not depend on the transport.** Even over CrypTen's own `gloo` transport, PriX is 3.14× faster than CrypTen on Transformer blocks on CPU (2.50–3.73× across modes) and 2.17× on GPU (1.94–2.47×), and 1.92× (CPU) / 2.17× (GPU) faster on primitives.
- **CrypTen's `gloo` path is slower than a lean TCP transport.** PriX's own TCP transport is 1.32× faster than `gloo` on CPU blocks (1.51× on primitives) and 1.07× faster on GPU

Table 6: **PriX speedup over CrypTen by PriX transport, on CPU and GPU** (geometric means over the 5 Transformer blocks or the 10 primitives of each mode; same sessions as Tables 2 and 4). CrypTen always uses `gloo` over TCP. "gloo" runs PriX over that same transport.

	Transformer blocks			**Primitives**		
Platform / Mode	TCP (default)	gloo	Shared mem.	TCP (default)	gloo	Shared mem.
CPU (AMD EPYC 7B12)						
CrypTen approx.	3.58×	2.50×	3.76×	2.14×	1.35×	2.88×
Piecewise poly	4.25×	3.21×	4.98×	2.63×	1.59×	3.08×
SHAFT	4.51×	3.73×	5.22×	1.93×	1.48×	2.89×
Curl LUTs	3.81×	2.77×	3.35×	3.70×	2.05×	4.13×
Fission	4.66×	3.71×	4.52×	5.00×	3.97×	5.17×
All modes (CPU)	**4.14**×	**3.14**×	**4.31**×	**2.89**×	**1.92**×	**3.53**×
GPU (NVIDIA Tesla T4)						
CrypTen approx.	2.21×	1.96×	2.32×	2.18×	1.74×	2.33×
Piecewise poly	2.16×	1.94×	2.33×	2.26×	1.77×	2.37×
SHAFT	2.06×	2.08×	2.35×	2.19×	1.75×	2.45×
Curl LUTs	2.80×	2.47×	2.87×	3.06×	2.35×	3.07×
Fission	2.51×	2.46×	2.47×	4.26×	3.84×	4.54×
All modes (GPU)	**2.33**×	**2.17**×	**2.46**×	**2.69**×	**2.17**×	**2.85**×

blocks (1.24× on primitives), so part of CrypTen's latency is its transport: `gloo` adds tensor conversions and collective-library overhead to every opening.

- **Shared memory adds little over TCP.** PriX over shared memory is only 4–5% faster than over TCP on blocks (4.31× vs. 4.14× on CPU; 2.46× vs. 2.33× on GPU), and 22% (CPU) / 6% (GPU) faster on the small, round-bound primitives, where per-round latency dominates. For openings above 16 MiB, TCP is faster, because the shared-memory transport falls back to pickled queues (46 vs. 162 ms for a 19 MB opening). We therefore made TCP the default; shared memory remains available as a single-host shortcut.

- **Fission mode depends least on the transport**, because it needs very few rounds between the MPC parties (about 14 per GPT-2 block, versus 98–176 in the other modes).

- **Communication is similar, but not identical.** In `crypten` mode, the two frameworks' round counts agree to within one round and their byte counts to within 5% on every Transformer block (e.g. 176 rounds and 80.1 vs. 79.7 MB for a GPT-2 block). Two PriX protocol changes account for most of the remaining differences:

  – *Smaller comparison openings.* In each carry-lookahead round, one masked vector P_0 is ANDed with both S_1 and P_1. CrypTen first copies P_0 to the shape of $[S_1, P_1]$ and therefore masks and opens it twice (its source marks this with a TODO); PriX uses a triple $(a, [b_1, b_2], [a \wedge b_1, a \wedge b_2])$ and opens $P_0 \oplus a$ once, as matrix Beaver triples do for a shared operand. The opened values are still uniformly masked by fresh triples, so security is unchanged. This cuts a comparison from 29 to 23 words per element (ReLU: 0.38 vs. 0.48 MB), and `poly`-mode blocks send 5–16% fewer bytes (GPT-2: 88.8 vs. 96.4 MB).

  – *Batched range tests in SHAFT.* PriX obtains $[x < 0]$ and $[|x| < w]$ from one comparison over the stacked tensor $[x, x-w, -x-w]$, while the SHAFT reference runs two sequential

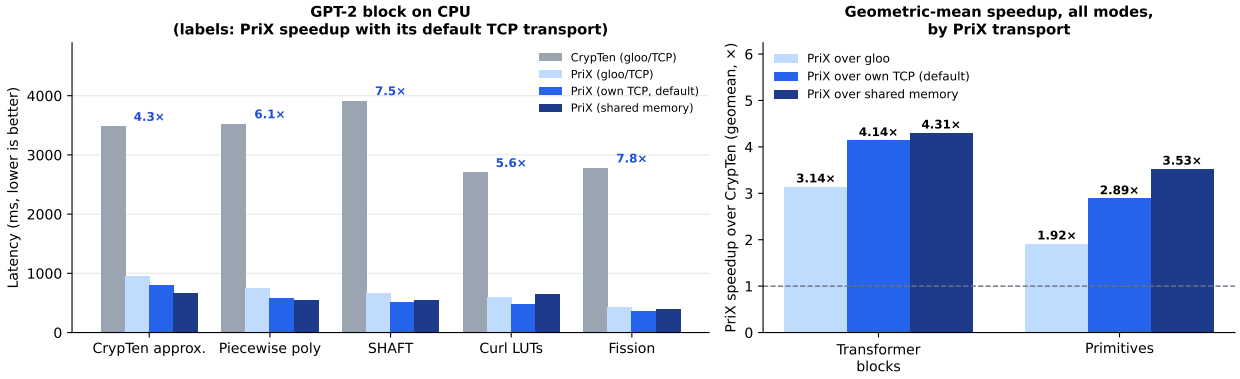


Figure 5: Transport comparison on CPU. Left: GPT-2 block latency for CrypTen and for PriX over `gloo`, its own TCP transport (default), and shared memory; labels give PriX's speedup with its default transport. Right: geometric-mean speedup over all modes for each PriX transport.

comparisons. This saves 8 rounds per GeLU/SiLU (11 vs. 19) and per SHAFT-mode block (GPT-2: 98 vs. 106), at the cost of about 4% more bytes.

PriX takes more rounds in two places, both because it applies a configuration setting that the CrypTen code ignores. In `lut` mode, Curl truncates squares with the communication-free local shift even though the mode specifies TruncPr for products (its multiplications do use TruncPr); PriX uses TruncPr for both, so its LayerNorm takes one extra round and each block two (GPT-2: 113 vs. 111). In `crypten` mode, CrypTen's sigmoid sets exp to 9 squarings in a configuration override that does not enclose the exp call, so it runs with the default 8; PriX applies the override, so its SiLU takes 25 rounds versus 24 and is about twice as accurate (Section 5.5). Apart from `shaft` mode, PriX's speedups therefore do not come from fewer rounds.

We also recorded the time spent inside communication calls, but for PriX this does not separate cleanly from computation. JAX dispatches device work asynchronously, and the copy to the host at the start of each communication call waits for that work to finish, so compute time is attributed to communication. The transport swap above avoids this problem because it compares end-to-end latencies.

5.5 Approximation Accuracy and Valid Input Ranges

The latency results above say nothing about whether each mode computes the right function. Table 7 reports the error of each nonlinearity in PriX's default configuration of each mode, measured on CPU with two parties against exact float64 results (`benchmarks/accuracy_sweep.py`). Each function is evaluated 10 times on a fixed grid of 241–257 points (softmax: $4 \times 32 \times 32$ logits drawn from $\mathcal{N}(0, 2^2)$; LayerNorm: $4 \times 32 \times 64$ inputs from $\mathcal{N}(0.5, 2^2)$), because probabilistic truncation and the masks make the error vary between runs.

No mode is accurate everywhere, and inputs outside a mode's valid range are neither clamped nor flagged:

- **`lut` (wavelet LUTs).** Curl's original defaults cover narrow ranges ($[0, 2)$ for $1/\sqrt{x}$ and $|x| < 4$ for GeLU, which caused LayerNorm to wrap whenever a row's variance was 2 or more and GeLU to wrap for $|x| \geq 4$) and suffer from an intermittent Softmax failure (≈ -461

Table 7: **Approximation error by mode** (default configuration of each mode, 2 parties, 10 runs, CPU). Each cell is the median absolute error / the typical (median over runs) worst-case absolute error within one run. Bold marks a worst-case error above 0.1; † marks functions for which at least one run had an error above 1.

Function (inputs)	`crypten`	`poly`	`shaft`	`lut`	`fission`
exp, $[-8, 0]$	4e-4 / 1e-3	2e-3 / 0.011	4e-4 / 1e-3	3e-4 / **0.12**	4e-6
exp, $[-4, 2]$	1e-3 / 0.070	3e-3 / 0.011	1e-3 / 0.056	9e-3 / **7.4**†	5e-6
log, $[0.1, 50]$	0.020 / 0.033	0.020 / 0.032	0.020 / 0.033	1e-5 / **2.0**†	4e-6
$1/x$, $[0.1, 50]$	9e-5 / 6e-4	9e-5 / 6e-4	9e-5 / 6e-4	7e-6 / **1.6e4**†	3e-6
$1/\sqrt{x}$, $[0.05, 50]$	2e-5 / **0.24**	2e-5 / **0.24**	2e-5 / **0.24**	**0.13** / **193**†	4e-6
Sigmoid, $[-8, 8]$	2e-4 / 2e-3	1e-3 / 9e-3	0.019 / **0.50**	5e-5 / 4e-3	4e-6
Tanh, $[-6, 6]$	1e-4 / 4e-3	1e-3 / 0.018	0.035 / 0.093	2e-5 / 9e-3	3e-6
Erf, $[-4, 4]$	**0.24** / **7.2e4**†	2e-3 / 0.015	0.027 / 0.071	5e-5 / 0.011	3e-6
GeLU, $[-6, 6]$	**0.92** / **1.4e6**†	4e-4 / 4e-3	4e-4 / 4e-3	3e-4 / **0.13**	3e-6
SiLU, $[-8, 8]$	6e-4 / 2e-3	5e-3 / 0.015	6e-4 / 5e-3	3e-3 / **0.32**	4e-6
Softmax	1e-4 / 4e-3	6e-4 / 6e-3	3e-4 / 0.052	1e-4 / 0.015	4e-6
LayerNorm	9e-6 / 9e-5	9e-6 / 8e-5	1e-5 / 1e-4	1e-5 / 9e-5	4e-6

on 3–4% of calls in both PriX and the Curl reference). Tracing the wavelet tables explains the Softmax failure: Biorthogonal-2.2's decomposition filter has negative side lobes ($-1/8$), so the $1/x$ pole at $x = 2^{-16}$ gives bin $k = 1$ of the reciprocal table a large negative knot (≈ -4968). With Curl's 2^7-bin reciprocal table on $[0, 64)$, bin $k = 1$ spans $[0.5, 1.0)$, while Curl's 2^5-bin negative-exponential table on $[0, 64)$ evaluates $\widehat{\exp}(0) = 0.924 < 1$. Whenever a Softmax row has one dominant logit, the sum of exponentials falls in $[0.924, 1.0)$ and lands in bin $k = 1$ whenever `TruncPr` rounds down. PriX therefore sets `inv_sqrt.lut_max_bits` $= 4$ ($[0, 16)$), `gelu.lut_max_bits` $= 3$ ($[-8, 8]$), `exp.lut_max_bits` $= 4$ ($\widehat{\exp}(0) = 0.992$), and `reciprocal.bior_size_bits` $= 8$ (moving $[0.75, 1.0)$ to bin $k = 3$), which fixes LayerNorm, GeLU on $[-6, 6]$, and Softmax without changing the table sizes or the communication volume (the latency benchmarks above use these defaults in both frameworks). Even with these defaults, $1/\sqrt{x}$ wraps for $x \geq 16$, exp assumes non-positive inputs (`all_neg = True`), and $1/x$, $1/\sqrt{x}$, and log remain inaccurate in their first two bins near 0 because of the Biorthogonal-2.2 boundary sidelobe.

- `crypten`. Erf and GeLU use an 8-term Taylor series that is accurate to 3×10^{-4} on $|x| \leq 1.75$ (GeLU: 10^{-4} on $|x| \leq 2.0$) but diverges for $|x| \gtrsim 2.5$, as in CrypTen. The Newton–Raphson $1/\sqrt{x}$ loses accuracy near the bottom of its range.

- `shaft`. The Fourier series are periodic, so Sigmoid and Tanh oscillate near the edge of the period ($|x| \approx 8$ by default).

- `poly` stays within about 0.02 on its fitted intervals, and `fission` is exact to float32 precision (its column shows the median error only).

Comparison with the CrypTen/Curl reference. On identical inputs (10 runs each), PriX is more accurate than the reference implementation on three functions, because of three bugs in the reference code rather than any protocol difference. (1) CrypTen's `var()` inverts its `unbiased` flag and divides by $n-1$ when asked for the biased variance, so every non-`fission` LayerNorm output is scaled by $\sqrt{(n-1)/n}$ (0.9922 for $n = 64$; a least-squares fit of Curl's outputs gives 0.9921). Curl's `lut`-mode LayerNorm has a median error of 5.4×10^{-3} versus 1.0×10^{-5} for PriX; the bias shrinks

with width (about 0.07% at $d = 768$). (2) The ineffective exp override described in Section 5.4 roughly doubles the error of CrypTen's `crypten`-mode SiLU (median 1.1×10^{-3} versus 5.6×10^{-4}). (3) CrypTen's integer `pow(p)` computes $(x^3)^{(p-1)/2} = x^{3(p-1)/2}$ instead of $(x^2)^{(p-1)/2} \cdot x = x^p$ for odd $p \geq 5$, which replaces odd powers x^5, x^9, x^{13}, x^{17} in its Taylor `erf` with even powers $x^6, x^{12}, x^{18}, x^{24}$ (raising its `erf` error on $[-1.5, 1.5]$ from 3×10^{-4} to 3.3 and its median error on $[-4, 4]$ from 0.24 to 2.5×10^3). None of these bugs changes the number of multiplications or the cost of the computation, so the latency comparison is unaffected. While preparing this evaluation, we found and fixed a bug in PriX's `fission` mode: when the right operand of a multiplication still carried a deferred truncation, the product was labelled with the wrong fixed-point scale, which made attention scores about 2^{16} times too small and attention nearly uniform. The fix changes only the scale label, not the protocol steps that run, so it does not affect the latencies above, which were measured before it. The reference implementations of Curl and Fission in the CrypTen baseline do not have this bug. We have not measured end-to-end model accuracy.

6 Protocol Case Studies: Function Secret Sharing, Wavelets, and Truncation Trade-offs

Because PriX executes distinct cryptographic paradigms inside the same compiled JAX/XLA runtime, transport, and fixed-point representation, it enables controlled cross-paradigm comparisons that are otherwise confounded by differences in implementation language, hardware backend, or ring bitwidth. In this section, we use PriX to investigate three questions raised by recent literature: (1) How do Function Secret Sharing (FSS) protocols—**SIGMA** [10] and **Wave** [23]—compare against **SHAFT** [13] and **Curl** [24] when evaluated on an equal footing? (2) How does Wave's deterministic DPF/DCF LUT truncation (`lut="wave"`) compare against its probabilistic local-shift variant (`lut="wave_prob"`)? (3) Why does CrypTen's 0-round local product truncation (`prod="crypten"`) exhibit near-zero error on standard single-block benchmarks despite being known to fail catastrophically in general?

6.1 Function Secret Sharing in PriX: SIGMA and Wave

In addition to secret-shared carry-lookahead circuits and one-hot LUT correlations, PriX implements 2-party Function Secret Sharing [3, 4] in `prix/fss.py`, compiling PRG tree traversal (`jax.lax.fori_loop`) and Grotto prefix-parity evaluation [25] into fused XLA executables:

- **Faithful SIGMA Protocols (`mode="sigma"`, `lut="sigma"`)** [10]: Rather than running 6 interactive rounds of Kogge–Stone AND gates for $\langle x \rangle^A < 0$, parties mask the input $\hat{x} = x + r$ in 1 online round (8 B) and locally evaluate a Grotto Distributed Point Function (DPF) prefix-parity tree to obtain XOR shares of the comparison bit $\langle x < 0 \rangle^B$, followed by 1 round of binary-to-arithmetic conversion or multiplexed selection (2 online rounds total, versus 8 rounds in SPK). For nonlinearities, `mode="sigma"` implements SIGMA's exact two-phase protocols from [10] (Figures 7–9, Table 2):

  1. *Symmetry, clipping, and Truncate-Reduce ($\Pi^{TR}_{n,f}$) before LUT lookup:* Because raw 0th-order lookup tables scale exponentially with bitwidth, SIGMA first executes online DPF comparisons and Π^{Select} multiplexers to fold negative inputs and clamp the domain before calling a small table, followed by a 1-round faithful Truncate-Reduce ($\Pi^{\text{TR}}_{n,f}$, Theorem 4) opening to drop low fractional bits before the 2-round DPF-LUT evaluation (3 rounds, 32 B/element per LUT). Specifically, $\text{GeLU}(x) =$

$\mathrm{ReLU}(x) - \mathrm{LUT}_{256}(\mathrm{Clip}(|x|, 0, 4))$ and $\mathrm{SiLU}(x) = \mathrm{ReLU}(x) - \mathrm{LUT}_{1024}(\mathrm{Clip}(|x|, 0, 8))$ require **6 online rounds and** 56.3 **B/element** (matching SIGMA Table 2's 58 B); $\exp(-x)$ (Figure 8) clips to $[0, 16)$, runs Π^{TR}, evaluates two 2^8-entry tables $\exp(-z_{\mathrm{high}}) \cdot \exp(-z_{\mathrm{low}})$ via a Beaver multiplication, and muxes clipped inputs (**12 rounds,** 120.1 **B/element**); and $1/\sqrt{x}$ (Figure 9) uses tailored range checks plus aLUT + Newton–Raphson refinement (**10 rounds,** 104.1 **B/element**, matching SIGMA Table 2's 106 B).

- **Deterministic Wave (`mode="wave"`, `lut="wave"`)** [23]: Wave bridges FSS and wavelet-compressed lookup tables in two ways: (i) because wavelet basis functions are compactly supported on $[-2^{\max}, 2^{\max})$, Wave encodes the entire activation (including sign and tail) directly into a single wavelet coefficient table, eliminating SIGMA's pre-LUT sign/clip comparisons, multiplexers, and Π^{TR} round; and (ii) it replaces 0th-order step tables with 1st-order piecewise-linear Biorthogonal bior$(5, 3)$ (LeGall) splines (and Haar bior$(1, 1)$ where appropriate), shrinking table sizes by 16×–64×. For a table of 2^J coarse intervals with step size $2^{\mathfrak{j}}$, parties open $\hat{x} = x + r \bmod 2^{J+\mathfrak{j}}$ in Round 1 (8 B) and locally evaluate:

  1. a $(J + \mathfrak{j})$-*level Grotto DPF* (`grotto_segment_parities_jit`) that outputs exact XOR shares of the one-hot indicator $\langle \mathbf{e}_k \rangle^B \in \{0, 1\}^{2^J}$ for the coarse bin $k = \lfloor x/2^{\mathfrak{j}} \rfloor$ *without* a separate Π^{TR} round and without off-by-1 probabilistic truncation error at interval boundaries; and
  2. a $\mathfrak{j}$-*level arithmetic Distributed Comparison Function (DCF)* (`dcf_eval_arith_jit`) that computes the exact low-bit borrow $\langle \mathbb{I}[\hat{x}_{\mathrm{low}} < r_{\mathrm{low}}] \rangle^A$ in 0 extra rounds, yielding exact additive shares of the intra-bin offset $\langle x_{\mathrm{low}} \rangle^A = \hat{x}_{\mathrm{low}} - \langle r_{\mathrm{low}} \rangle^A - 2^{\mathfrak{j}} \langle \mathbb{I}[\hat{x}_{\mathrm{low}} < r_{\mathrm{low}}] \rangle^A$.

  In Round 2, Wave's $\Pi^{\text{LUT-bior}(5,3)}$ protocol (`wave_bior_lut`) uses a generalized Beaver 8-tuple $(\tau_0, \tau_1, \theta, \alpha_0, \alpha_1, \beta_0, \beta_1, \gamma)$ to simultaneously convert the XOR-shared slope and intercept (v_0, v_1) into arithmetic shares over $\mathbb{Z}_{2^{64}}$ and multiply $v_0 \cdot \langle x_{\mathrm{low}} \rangle^A$ in a single 32 B opening. This evaluates a piecewise-linear wavelet LUT in **2 online rounds and** 40 **B/element**—cutting SIGMA's 6–12 rounds to 2, halving Curl's 4 rounds, and eliminating boundary jump errors.

- **Probabilistic Wave (`lut="wave_prob"`)**: Following Wave §5.3, PriX also supports a probabilistic truncation variant (`_wave_bior_prob_r1_jit`) that locally splits $\langle x \rangle^A$ in 0 rounds via

$$x_{\mathrm{high},b} = (x_b \gg \mathfrak{j}) + \mathbb{I}[b = 0], \qquad x_{\mathrm{lsb},b} = \big(x_b \,\&\, (2^{\mathfrak{j}} - 1)\big) - \mathbb{I}[b = 0] \cdot 2^{\mathfrak{j}}, \tag{6}$$

  which preserves the exact identity $x_{\mathrm{high}} \cdot 2^{\mathfrak{j}} + x_{\mathrm{lsb}} = x$ while guaranteeing $x_{\mathrm{high}} \in \{k, k + 1\}$ (preventing bin 0 from wrapping to $2^J - 1$). Because x_{high} is already truncated to J bits, parties evaluate only a J-level DPF with no $\mathfrak{j}$-level DPF tail and no arithmetic DCF.

6.2 Re-evaluating SHAFT vs. SIGMA, Wave, and Curl under a Unified Runtime

SHAFT [13] (Table VII) reported 3.7×–6.4× lower GeLU latency and 1.4×–2.5× lower end-to-end Transformer latency than SIGMA [10], attributing these gains to replacing lookup tables with 1-round Fourier series and ODE Softmax. However, that comparison benchmarked SHAFT's Python/CrypTen code at $f = 16$ against SIGMA's standalone C++/CUDA binary over 37–50-bit rings. Tables 8 and 9 compare **Wave** (`mode="wave"`), **SIGMA** (`mode="sigma"`), **Curl** (`mode="lut"`), and **SHAFT** (`mode="shaft"`) inside PriX on CPU across primitive nonlinearities, `BERT-base` sublayers ($L = 128$), and full 12- and 24-layer Transformer models.

Table 8: **Cross-paradigm comparison of Wave, SIGMA, Curl, and SHAFT on primitive nonlinearities ($N = 1000$ elements)** in PriX (2 parties, CPU, $f = 16$). We report online communication rounds, per-party online communication (bytes/element), local CPU execution time (ms), and mean absolute error (MAE) against float64.

Primitive (Range)	Metric	Wave (`mode="wave"`)	SIGMA (`mode="sigma"`)	Curl (`mode="lut"`)	SHAFT (`mode="shaft"`)
GeLU ($[-3.8, 3.8]$)	Rounds / Comm (1P) Time / MAE	**2 rds** / **40.0** B **10.5** ms / 1.75×10^{-3}	6 rds / 56.3 B 28.8 ms / $\mathbf{3.49 \times 10^{-4}}$	4 rds / **40.0** B 15.1 ms / 9.07×10^{-3}	11 rds / 544.0 B 15.5 ms / 9.41×10^{-4}
SiLU ($[-7.5, 7.5]$)	Rounds / Comm (1P) Time / MAE	**2 rds** / **40.0** B 7.9 ms / 3.47×10^{-3}	6 rds / 56.3 B 32.4 ms / $\mathbf{4.96 \times 10^{-4}}$	4 rds / **40.0** B **7.2** ms / 1.69×10^{-2}	11 rds / 544.0 B 23.1 ms / 8.55×10^{-4}
Sigmoid ($[-7.5, 7.5]$)	Rounds / Comm (1P) Time / MAE	6 rds / 80.3 B 36.8 ms / $\mathbf{7.24 \times 10^{-5}}$	7 rds / 72.3 B 26.5 ms / 4.16×10^{-3}	15 rds / 592.0 B 21.7 ms / 3.70×10^{-4}	**1 rd** / **8.0** B **3.2** ms / 2.24×10^{-2}
Tanh ($[-4.5, 4.5]$)	Rounds / Comm (1P) Time / MAE	6 rds / 80.3 B 32.0 ms / $\mathbf{1.17 \times 10^{-4}}$	7 rds / 72.3 B 21.3 ms / 6.93×10^{-3}	15 rds / 592.0 B 23.2 ms / 5.24×10^{-4}	**1 rd** / **8.0** B **1.8** ms / 3.03×10^{-2}
Erf ($[-3.5, 3.5]$)	Rounds / Comm (1P) Time / MAE	9 rds / 104.3 B 37.6 ms / $\mathbf{1.68 \times 10^{-4}}$	10 rds / 96.3 B 34.3 ms / 8.93×10^{-3}	23 rds / 424.0 B 29.0 ms / 8.29×10^{-4}	**1 rd** / **8.0** B **2.8** ms / 2.37×10^{-2}
nexp ($\exp(x), [-15, 0)$)	Rounds / Comm (1P) Time / MAE	**5 rds** / 64.1 B **12.3** ms / 6.04×10^{-4}	12 rds / 120.1 B 38.7 ms / $\mathbf{1.13 \times 10^{-5}}$	13 rds / 224.0 B 13.0 ms / 2.24×10^{-3}	8 rds / **64.0** B 12.5 ms / 3.09×10^{-4}
$1/x$ ($[1.0, 30.0]$)	Rounds / Comm (1P) Time / MAE	**2 rds** / 40.0 B 31.6 ms / 8.51×10^{-5}	3 rds / **32.0** B 20.7 ms / 5.36×10^{-4}	4 rds / 40.0 B **10.0** ms / 3.42×10^{-4}	28 rds / 304.0 B 56.2 ms / $\mathbf{8.08 \times 10^{-5}}$
$1/\sqrt{x}$ ($[0.1, 15.0]$)	Rounds / Comm (1P) Time / MAE	**2 rds** / **40.0** B 97.1 ms / $\mathbf{2.61 \times 10^{-5}}$	10 rds / 104.1 B 45.8 ms / 2.25×10^{-4}	4 rds / **40.0** B **15.6** ms / 1.03×10^{-4}	17 rds / 184.0 B 22.0 ms / 4.65×10^{-4}

Three findings emerge from Tables 8 and 9:

1. **Disentangling SHAFT vs. SIGMA:** First, our `mode="sigma"` implementation reproduces SIGMA's reported per-party communication [10] (Tables 2, 3, 5, and 8): 56.3 B/element on GeLU / SiLU (vs. 58 B in SIGMA), 32.0 B on $1/x$ (vs. 36 B), 104.1 B on $1/\sqrt{x}$ (vs. 106 B), 0.265 GB per party across 12 `BERT-base` GeLU layers (vs. 0.26 GB in SIGMA Table 3), 0.341 GB across 12 Softmax layers (vs. 0.37 GB), and end-to-end 1-party model communication matching within 0.6% (**0.984** GB vs. **0.99** GB on `BERT-base`; **2.624** GB vs. **2.64** GB on `BERT-large`; **0.393** GB vs. **0.37** GB on `GPT-2` at $L = 64$). Comparing SHAFT against SIGMA on an equal footing reveals why their relative performance depends on the network regime: SHAFT's Fourier series requires 1 online round only when the input is guaranteed to stay within one half-period (Sigmoid, Tanh, Erf). For unbounded activations (GeLU and SiLU), SHAFT must first evaluate secure range-clipping comparisons ($[x < 0]$ and $[|x| < w]$), requiring **11 online rounds and** 544 **B/element** (427.8 MB on `BERT-base` GeLU) compared with **6 rounds and** 56.3 **B/element** (44.2 MB) for SIGMA and **2 rounds and** 40.0 **B/element** (31.5 MB) for Wave. Across a 12-layer `BERT-base` ($L = 128$), SHAFT requires **1,200 online rounds and** 11.02 **GB**, whereas SIGMA requires **1,032 rounds and** 1.97 **GB** (5.6× less communication) and Wave requires **696 rounds and** 1.55 **GB** (1.72× fewer rounds and 7.1× less communication). Why is SHAFT faster than SIGMA in local CPU runtime inside JAX (20.06 s vs. 72.41 s)? Because XLA does not expose x86 hardware `AES-NI` intrinsics (`_mm_aesenc_si128`) that C++ FSS engines use to evaluate 128-bit DPF nodes in a single hardware instruction; instead, JAX evaluates our 64-bit counter-mode PRG over 256- to 1024-leaf DPF trees per element (2.91 s on GeLU, 32.4 ms on SiLU), whereas SHAFT evaluates a dense linear combination after its comparison rounds (0.51 s). Over any wide-area network (100 Mbps–1 Gbps bandwidth, 10–40 ms RTT), SHAFT's 9.0–9.5 GB of extra traffic adds 75–750 + s of network transfer time, reversing its local-CPU advantage.

2. **Wave vs. SIGMA (why Wave's wavelet representation improves rounds, communication, and compute simultaneously):** Replacing SIGMA's multi-phase 0th-order LUT protocols with Wave's 2-round bior$(5, 3)$ wavelet LUTs wins on all three axes:

Table 9: **Sublayer ($L = 128$) and end-to-end Transformer comparison of Wave, SIGMA, Curl, and SHAFT** in PriX (2 parties, CPU, $f = 16$). Total 2P Comm reports combined online traffic sent by both parties.

Benchmark	Metric	Wave (`mode="wave"`)	SIGMA (`mode="sigma"`)	Curl (`mode="lut"`)	SHAFT (`mode="shaft"`)
GeLU $(1, 128, 3072)$	Rounds / 2P Comm	**2 rds** / **31.46** MB	6 rds / 44.24 MB	4 rds / **31.46** MB	11 rds / 427.82 MB
	Local Time (s)	1.77 s	2.91 s	**0.10** s	0.51 s
	MAE / Max Error	0.00190 / 0.0158	**0.00038** / **0.0039**	0.00979 / 0.2096	0.00098 / 0.0045
Softmax $(1, 12, 128, 128)$	Rounds / 2P Comm	**23 rds** / **34.82** MB	31 rds / 56.82 MB	98 rds / 311.87 MB	41 rds / 346.03 MB
	Local Time (s)	1.61 s	2.46 s	0.64 s	**0.55** s
	MAE / Max Error	6.78×10^{-5} / 0.00093	$\mathbf{8.68 \times 10^{-6}}$ / **0.00012**	2.81×10^{-4} / 0.00715	2.41×10^{-4} / 0.00522
LayerNorm $(1, 128, 768)$	Rounds / 2P Comm	**6 rds** / 6.30 MB	14 rds / 6.32 MB	8 rds / 6.30 MB	19 rds / **3.19** MB
	Local Time (s)	0.043 s	0.062 s	**0.030** s	0.031 s
	MAE / Max Error	2.98×10^{-5} / **0.00020**	1.13×10^{-3} / 0.00961	$\mathbf{2.80 \times 10^{-5}}$ / 0.00041	8.48×10^{-4} / 0.00551
`BERT-base` (12L, $L = 128$)	Rounds / Block (Total)	**58** (**696**)	86 (1,032)	139 (1,668)	100 (1,200)
	Total 2P Comm (GB)	**1.551** GB	1.968 GB	6.234 GB	11.024 GB
	Local Runtime (12L)	43.00 s	72.41 s	**17.79** s	20.06 s
	Block MAE	0.0434	0.0445	0.0506	**0.0015**
`BERT-large` (24L, $L = 128$)	Rounds / Block (Total)	**58** (**1,392**)	86 (2,064)	139 (3,336)	100 (2,400)
	Total 2P Comm (GB)	**4.135** GB	5.248 GB	17.833 GB	30.605 GB
	Local Runtime (24L)	115.33 s	213.66 s	**44.18** s	56.75 s
	Block MAE	0.0423	0.0433	0.0500	**0.0015**
`GPT-2` (12L, $L = 64$)	Rounds / Block (Total)	**56** (**672**)	84 (1,008)	138 (1,656)	100 (1,200)
	Total 2P Comm (GB)	**0.643** GB	0.786 GB	2.782 GB	5.144 GB
	Local Runtime (12L)	21.56 s	33.85 s	**9.77** s	11.01 s
	Block MAE	0.0028	0.0017	0.0190	**0.0015**

- 1.48×–5× *fewer rounds:* Because wavelet basis functions are compactly supported, Wave encodes the full activation domain directly into a single wavelet LUT and folds exact bin truncation into the $(J + \mathfrak{j})$-level Grotto DPF + $\mathfrak{j}$-level arithmetic DCF. This eliminates SIGMA's pre-LUT sign/clip comparisons, multiplexers, and Π^{TR} round—cutting GeLU / SiLU from 6 → 2 rounds, nexp from 12 → 5 rounds, $1/\sqrt{x}$ from 10 → 2 rounds, Softmax from 31 → 23 rounds, LayerNorm from 14 → 6 rounds, and a full Transformer block from 86 → 58 **rounds** $(1,032 \rightarrow 696$ **on** `BERT-base`**)**.
- 21%–62% *lower communication:* Wave cuts per-element traffic from 56.3 → 40.0 B on GeLU / SiLU (−29%), 120.1 → 64.1 B on nexp (−47%), 104.1 → 40.0 B on $1/\sqrt{x}$ (−62%), 56.8 → 34.8 MB on Softmax (−39%), and 1.968 → 1.551 **GB on 12-layer** `BERT-base` **(**−21%**)** (5.248 → 4.135 GB on `BERT-large`).
- 1.5×–4.1× *faster local CPU compute:* Because 1st-order piecewise-linear bior$(5, 3)$ splines represent smooth curves with 16×–64× **fewer LUT intervals** than SIGMA's 0th-order step tables ($2^4 = 16$ vs. $2^8 = 256$ knots on GeLU, $2^4 = 16$ vs. $2^{10} = 1024$ on SiLU, and one $2^5 = 32$ table vs. two $2^8 = 256$ tables on nexp), Wave's Grotto DPF prefix tree evaluates 16×–64× fewer leaves per element—accelerating GeLU by 1.6× (1.77 s vs. 2.91 s), SiLU by 4.1× (7.9 ms vs. 32.4 ms), Softmax by 1.5× (1.61 s vs. 2.46 s), and end-to-end `BERT-base` / `BERT-large` by 1.7×–1.9× (43.0 s vs. 72.4 s; 115.3 s vs. 213.7 s).

3. **Wave vs. Curl:** Compared with Curl (`mode="lut"`), Wave reduces Transformer rounds by 2.4× (58 vs. 139 rounds/block; 23 vs. 98 on Softmax) and total online communication by 4.0×–4.3× (1.55 GB vs. 6.23 GB on `BERT-base`), while improving accuracy on every operation (GeLU max error drops 13× from 0.2096 to 0.0158; `GPT-2` block MAE drops 6.8× from 0.0190 to 0.0028) because deterministic DPF/DCF truncation eliminates off-by-1 knot boundary jumps.

Table 10: **Ablation of deterministic vs. probabilistic LUT truncation (`lut="wave"` vs. `"wave_prob"`) and interactive vs. local product truncation (`prod="egk"` vs. `"crypten"`)** in Wave mode (2 parties, CPU, $f = 16$).

Benchmark	Metric	`wave_det_egk` (`lut="wave"`, `prod="egk"`)	`wave_det_crypten` (`lut="wave"`, `prod="crypten"`)	`wave_prob_egk` (`lut="wave_prob"`, `prod="egk"`)	`wave_prob_crypten` (`lut="wave_prob"`, `prod="crypten"`)
GeLU $(1, 128, 3072)$	Rounds / 2P Comm	2 rds / 31.46 MB	2 rds / 31.46 MB	2 rds / 31.46 MB	2 rds / 31.46 MB
	Time / MAE / Max	1.862 s / **.00190** / **.0158**	2.371 s / **.00190** / **.0158**	0.249 s / .00977 / .2091	**0.195** s / .00977 / .2084
Softmax $(1, 12, 128, 128)$	Rounds / 2P Comm	23 rds / 34.82 MB	**22 rds** / **31.68** MB	23 rds / 34.82 MB	**22 rds** / **31.68** MB
	Time / MAE / Max	2.010 s / **6.8e-5** / **.0009**	2.133 s / **6.8e-5** / **.0009**	0.752 s / 2.8e-4 / .0075	**0.734** s / 2.8e-4 / .0081
LayerNorm $(1, 128, 768)$	Rounds / 2P Comm	6 rds / 6.30 MB	**4 rds** / **3.16** MB	6 rds / 6.30 MB	**4 rds** / **3.16** MB
	Time / MAE / Max	0.054 s / **3.0e-5** / **.0002**	0.031 s / **3.0e-5** / **.0002**	0.028 s / 4.0e-5 / .0005	**0.024** s / 4.0e-5 / .0004
`BERT-base` (12L, $L = 128$)	Rounds / 2P Comm	696 rds / 1.551 GB	**504 rds** / **1.135** GB	696 rds / 1.551 GB	**504 rds** / **1.135** GB
	Total Time (12L)	51.32 s	50.19 s	16.89 s	**15.36** s
	Block MAE / Max	**0.0434** / 0.2238	**0.0434** / **0.2237**	0.0506 / 0.2559	0.0506 / 0.2504
`BERT-large` (24L, $L = 128$)	Rounds / 2P Comm	1,392 rds / 4.135 GB	**1,008 rds** / **3.028** GB	1,392 rds / 4.135 GB	**1,008 rds** / **3.028** GB
	Total Time (24L)	146.35 s	140.91 s	47.94 s	**44.45** s
	Block MAE / Max	**0.0423** / **0.1854**	**0.0423** / **0.1854**	0.0498 / 0.2564	0.0499 / 0.2499
`GPT-2` (12L, $L = 64$)	Rounds / 2P Comm	672 rds / 0.643 GB	**480 rds** / **0.464** GB	672 rds / 0.643 GB	**480 rds** / **0.464** GB
	Total Time (12L)	26.89 s	25.91 s	8.20 s	**6.96** s
	Block MAE / Max	**0.0028** / 0.0155	**0.0028** / **0.0154**	0.0191 / 0.1103	0.0191 / 0.1080

6.3 Deterministic vs. Probabilistic Truncation: Product Shift vs. LUT Indexing

In mixed-mode wavelet protocols such as Wave and Curl, fixed-point truncation occurs in two distinct contexts: (1) *product truncation* (`encoder.trunc_method.prod`), which shifts a 2^{2f}-scaled product $Z = \lfloor x \cdot y \cdot 2^{2f} \rceil$ back to scale 2^f via interactive TruncPr (`"egk"`) or 0-round local share shift (`"crypten"`); and (2) *LUT index truncation* (`encoder.trunc_method.lut`), which maps a secret input x to a coarse wavelet bin index $k = \lfloor x/2^{\mathsf{j}} \rfloor$ via deterministic DPF/DCF (`"wave"`) or 0-round probabilistic share splitting (`"wave_prob"`, Equation 6). Table 10 evaluates all four combinations in PriX.

Table 10 exposes a striking asymmetry between LUT truncation and product truncation:

1. Why probabilistic LUT truncation (`lut="wave_prob"`) speeds up local compute by 3.3×–9.6× but raises error by 5×–13×. Replacing deterministic DPF/DCF LUT truncation (`lut="wave"`) with probabilistic share splitting (`lut="wave_prob"`) removes both the j-level DPF carry loop ($\mathsf{j} = 12$ for GeLU) and the j-level arithmetic DCF, shrinking the DPF depth from $J + \mathsf{j} = 16$ levels to $J = 4$ levels. This accelerates local CPU GeLU by 9.6× (1.862 s → 0.195 s) and 12-layer `BERT-base` inference by 3.3× (51.32 s → 15.36 s), making `wave_prob_crypten` faster than both SHAFT (20.06 s) and Curl (17.79 s) even in pure local CPU runtime while using 3.3× **fewer rounds (504 vs. 1,668)** and 5.5× **less bandwidth (1.135 GB vs. 6.234 GB)** than Curl.

However, `lut="wave_prob"` increases GeLU MAE by 5.1× (0.00190 → 0.00977, Max Error 0.0158 → 0.2084) and `GPT-2` block MAE by 6.8× (0.0028 → 0.0191), matching Curl's TruncPr error (0.00979 on GeLU, 0.0190 on `GPT-2`). In a LUT protocol, the probabilistic 1-bit carry occurs on the *coarse bin index* $x_{\text{high}} = \lfloor x_0/2^{\mathsf{j}} \rfloor + \lfloor x_1/2^{\mathsf{j}} \rfloor + 1$: whenever $(x_0 \bmod 2^{\mathsf{j}}) + (x_1 \bmod 2^{\mathsf{j}}) < 2^{\mathsf{j}}$ (which occurs with probability $\approx 1/2$) and x lies near a wavelet knot boundary, x_{high} selects bin $k + 1$ instead of bin k—equivalent to perturbing the lookup address by $2^{\mathsf{j}} \in [1024, 4096]$ ULPs. In bior$(5, 3)$ piecewise-linear interpolation, the un-shifted low bits x_{lsb} are then multiplied by the *neighboring spline segment's slope and intercept* $(v_{0,k+1}, v_{1,k+1})$, introducing a boundary jump of magnitude $|\Delta v_0| \cdot 2^{\mathsf{j}-f}$ (up to ≈ 0.21 near GeLU's inflection points).

2. The Signed-Ring Paradox: Why `prod="crypten"` has near-zero error on small 2PC tensors yet is unsafe in general. Conversely, switching product truncation from interactive TruncPr (`prod="egk"`) to CrypTen's 0-round local shift (`prod="crypten"`) reduces Transformer rounds by 28% (58 → 42 rounds/block) and communication by 27% (1.551 GB → 1.135 GB on

`BERT-base`) with *zero measurable degradation in accuracy* (block MAE 0.0434 vs. 0.0434). Why does local share division appear so accurate here when it is widely known to be inaccurate in MPC?

The answer lies in the difference between unsigned rings $[0, 2^{64})$ and signed two's-complement rings $[-2^{63}, 2^{63})$:

- In an **unsigned ring** $[0, 2^{64})$, uniform additive shares $s_0, s_1 \in [0, 2^{64})$ of a small positive product $Z = \lfloor x \cdot y \cdot 2^{2f} \rceil > 0$ satisfy $s_0 + s_1 = Z + 2^{64}$ over $\mathbb{Z}$ with probability $1 - Z/2^{64} \approx 1$. Local unsigned right-shift $(s_0 \gg f) + (s_1 \gg f)$ therefore suffers a catastrophic $+2^{64-f}$ wrap-around error on almost every element.

- Because PriX and CrypTen represent 2-party shares as **signed two's-complement** `int64` in $[-2^{63}, 2^{63})$ centered at 0 and divide via signed arithmetic shift (`jax.lax.div`), two uniform signed shares $s_0, s_1 \in [-2^{63}, 2^{63})$ with $s_0 + s_1 \equiv Z \pmod{2^{64}}$ sum to Z *without overflow over* $\mathbb{Z}$ unless s_1 falls in the narrow boundary strip of width $|Z|$ at the edge of $[-2^{63}, 2^{63})$ where $Z - s_1$ overflows $[-2^{63}, 2^{63})$. Over the signed ring, local division decomposes into two error terms:

$$\left\lfloor \frac{s_0}{2^f} \right\rfloor + \left\lfloor \frac{s_1}{2^f} \right\rfloor = \left\lfloor \frac{Z}{2^f} \right\rfloor - \underbrace{\mathbb{I}\Big[(s_0 \bmod 2^f) + (s_1 \bmod 2^f) \geq 2^f\Big]}_{\text{LSB carry: at most 1 ULP } (2^{-f} \approx 1.5 \times 10^{-5})} \pm \underbrace{2^{64-f} \cdot \mathbb{I}[\text{MSB overflow}]}_{\text{Catastrophic spike: } \pm 2^{64-2f}}, \quad (7)$$

where the probability of the catastrophic $\pm 2^{64-2f} = \pm 2^{32}$ spike on a single multiplication is

$$\Pr[\text{catastrophic } \pm 2^{64-2f} \text{ spike}] = \frac{|Z|}{2^{64}} = \frac{|x \cdot y| \cdot 2^{2f}}{2^{64}}. \quad (8)$$

In our single-block benchmarks with $f = 16$ ($2^{2f} = 2^{32}$) and standard initialized weights/activations ($|x| \sim 0.4, |W| \sim 0.04$), a typical product has $|Z| \lesssim 2^{31}$, yielding a per-element wrap rate of $\approx 2^{-33} \approx 1.1 \times 10^{-10}$ ($\sim 0.03\%$ per Transformer block). Whenever no MSB overflow occurs, the 1-bit LSB carry perturbs the *output value* directly by $2^{-16} \approx 1.5 \times 10^{-5}$—identical to TruncPr—rather than shifting a coarse LUT bin index.

Despite its benign appearance on small single-block inputs, `prod="crypten"` fails in four critical regimes that motivate TruncPr (`"egk"`) and deterministic FSS truncation:

1. **Large intermediate activations or higher fixed-point precision f:** When dot products or hidden states grow (e.g. $|x \cdot y| \sim 10^3$ in deep unnormalized layers) or when precision is increased to $f = 20$–24, $|Z|/2^{64}$ increases by $10^3\times$–$10^5\times$. Across multi-layer, multi-batch inference, even a 10^{-6} per-element wrap probability produces $\pm 2^{32} \approx \pm 4.29 \times 10^9$ spikes that corrupt entire attention heads or output logits.

2. **Multi-party MPC ($N \geq 3$ parties):** For $N \geq 3$ parties, the sum of N uniform signed shares in $[-2^{63}, 2^{63})$ overflows $[-2^{63}, 2^{63})$ with probability $\geq 1/2$ on every element, requiring an interactive wrap-counting protocol (`_wraps` in `prix/array.py`).

3. **Chaining after modular-wrapping protocols:** Protocols like TruncPr (`_egk_pre_reveal` in `prix/array.py`) shift masked shares to bit 62 and back, producing output shares s_0, s_1 that wrap modulo 2^{64} with probability $1/2$ even for tiny values. Applying local `crypten` division to a share produced by such a protocol corrupts $\sim 50\%$ of all entries.

4. **Broken share uniformity in the upper f bits:** After local arithmetic right-shift by f, each party's share is restricted to $[-2^{63-f}, 2^{63-f})$ rather than the full ring $\mathbb{Z}_{2^{64}}$. Its upper f bits are deterministic sign-extensions ($s_0 \approx -s_1$), which breaks security if any subsequent protocol opens a shifted or partially masked share without fresh full-ring re-masking.

7 Security Considerations and Limitations

PriX is a research framework operating in the **semi-honest (honest-but-curious)** model, and its current implementation has the following limitations:

1. **Correlated Randomness Providers (`TFP` vs. `TTP`):** Like CrypTen [15], PriX defaults to the Trusted First Party (`TFP`) provider, in which Party 0 generates the Beaver triples, truncation pairs, LUT correlations, and FSS keys. Party 0 therefore knows the dealer randomness and can recover every secret; `TFP` is suitable only for performance measurement, and all experiments in this report use it for both frameworks. Setting `cfg.mpc.provider = "TTP"` runs a separate dealer process (`TTPServer`) under the usual non-colluding third-party assumption.

2. **Pseudorandom Number Generation:** By default, correlated randomness and FSS PRG trees are expanded with a unified counter-mode `SplitMix64` stream on both CPU and GPU (seeded from OS entropy exchanged at initialization and passed as traced kernel arguments rather than static compile-time constants). Because `SplitMix64` is not a cryptographic PRF, PriX also provides a keyed counter-mode **SipHash-2-4** cryptographic PRF [1] selectable via `cfg.mpc.prng = "siphash24"`.

3. **Probabilistic Truncation Range:** PriX's TruncPr [8] is instantiated with parameter $\ell = 62$ in $\mathbb{Z}_{2^{64}}$: the opened value is $2^{64-\ell-1}(x + 2^{\ell-1} + r)$ with a uniform ℓ-bit mask r and a masked carry bit. The opening hides x regardless of its value; the range condition $|x| < 2^{\ell-1} = 2^{61}$ on the ring integer is needed for *correctness*. With $f = 16$, this means real values below 2^{29} in magnitude for products at precision $2f$, and below 2^{45} for values at precision f. Values outside this range give incorrect results without any error being raised. As usual, the truncated result may be off by one unit in the last place.

4. **Implementation Hardening:** The `TTP` dealer enforces an RPC method allowlist, cached wavelet lookup tables and tuple caches are stored in per-user (`0700`) directories using pickle-free binary serialization, and both the peer-to-peer `tcp` transport (with optional mutual TLS) and the `jax.distributed` transport use PriX's pickle-free wire format. However, the `jax.distributed` key-value store itself does not provide channel encryption or authentication, and Party 0 drives `TTP` and evaluator control messages.

5. **Evaluation Scope:** All measurements are single-host, with no network emulation; they characterize computation and local inter-process overhead, not wide-area deployment. We report the accuracy of individual nonlinearities and Transformer blocks (Sections 5.5 and 6) but not downstream task accuracy on trained checkpoints. PriX has been tested on CPUs and CUDA GPUs only. On TPUs, its FP64 limb matrix multiplication would rely on emulated double precision.

8 Related Work

MPC frameworks for machine learning. CrypTen [15] is the PyTorch framework on which Curl, Fission, and our baseline build. MP-SPDZ [14] compiles programs for a large set of MPC protocols and security models, but targets general computation rather than tensor workloads. The closest system to PriX is SecretFlow-SPU [19], which also compiles machine-learning programs (including JAX) through XLA: SPU lowers the whole program to its own IR and executes it with a C++ runtime over several protocols, whereas PriX stays in Python, compiles only the local

phases between openings with JAX itself, and aims at fast prototyping of CrypTen-style and FSS approximation methods. PUMA [7] is implemented on SPU.

GPU acceleration and Function Secret Sharing. CryptGPU [26] runs CrypTen-style 3-party protocols on GPUs and introduced the floating-point limb decomposition for ring matrix multiplication that CrypTen's CUDA backend uses; PriX refines it by computing only the limb products that survive modulo 2^{64}. Piranha [29] is a general GPU platform for secret-sharing MPC. Building on Function Secret Sharing [3, 4] and Grotto's prefix-parity trees [25], SIGMA [10] evaluates Transformer nonlinearities via 2-round DPF lookup tables, while Wave [23] combines DPF/DCF evaluation with Haar and Biorthogonal bior$(5, 3)$ wavelet transforms to shrink LUT domains and achieve deterministic 2-round piecewise-linear evaluation; PriX implements both `sigma` and `wave` modes as compiled XLA kernels.

Private Transformer inference. Besides the seven paradigms PriX implements [15, 7, 21, 18, 13, 24, 10, 23, 27], two-party systems combining homomorphic encryption with secret sharing, such as Iron [11], BOLT [21], and BumbleBee [12], target the client–server setting with lower communication, and MPCFormer [17] replaces nonlinearities with MPC-friendly ones and recovers accuracy by distillation. These are complementary: PriX's contribution is a unified XLA execution engine and cross-paradigm benchmark suite for secret-sharing and FSS protocols.

9 Conclusion

We presented **PriX**, a JAX/XLA framework for privacy-preserving Transformer research via Secure Multi-Party Computation and Function Secret Sharing. PriX implements seven MPC approximation paradigms (**CrypTen**, **piecewise polynomials**, **SHAFT**, **Curl**, **SIGMA**, **Wave**, and **Fission**) behind one API. It compiles the local steps of each protocol with XLA, computes only the surviving limb products in its exact FP64 ring matrix multiplication, and communicates over a pickle-free peer-to-peer TCP transport that also runs across machines. On a single host, it has lower latency than the PyTorch CrypTen baseline in every configuration we measured, with Transformer-block speedups of **2.32**×–**7.77**× on CPU and **1.91**×–**3.08**× on a Tesla T4 GPU. Furthermore, our cross-paradigm case studies demonstrate that combining 1st-order bior$(5, 3)$ wavelets with Function Secret Sharing (**Wave**) reduces online rounds by 1.48× (58 vs. 86 rounds/block), online communication by 21%–62%, and local DPF evaluation cost by 3.3×–11.5× over multi-phase 0th-order FSS (**SIGMA**), cuts round complexity by 2.4× and bandwidth by 4.0× over one-hot wavelet LUTs (**Curl**), and cuts online communication by 7.1× over Fourier-series activations (**SHAFT**), while clarifying the distinct error and security regimes of deterministic versus probabilistic truncation in LUT indexing and ring multiplication. PriX is open-source and available on PyPI via `pip install prix-jax`.

References

[1] Jean-Philippe Aumasson and Daniel J. Bernstein. SipHash: A fast short-input PRF. In *International Conference on Cryptology in India (INDOCRYPT)*, volume 7668 of *LNCS*, pages 489–508, 2012.

[2] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology – CRYPTO '91*, pages 420–432. Springer, 1991.

[3] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367. Springer, 2015.

[4] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1292–1303, 2016.

[5] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax, 2018.

[6] Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In *Theory of Cryptography Conference (TCC)*, volume 3876 of *Lecture Notes in Computer Science*, pages 285–304, 2006.

[7] Ye Dong, Wen jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Chen. PUMA: Secure inference of LLaMA-7B in five minutes. *arXiv preprint arXiv:2307.12533*, 2023. https://arxiv.org/abs/2307.12533.

[8] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology – CRYPTO 2020*, volume 12171 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.

[9] Charles Gouert, Mehmet Ugurbil, Dimitris Mouris, Miguel de Vega, and Nektarios G. Tsoutsos. Ripple: Accelerating programmable bootstraps for FHE with wavelet approximations. In *Information Security – 27th International Conference (ISC 2024)*, volume 15155 of *Lecture Notes in Computer Science*, pages 273–293. Springer, 2024. Cryptology ePrint Archive, Paper 2024/866, https://eprint.iacr.org/2024/866.

[10] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. SIGMA: Secure GPT inference with function secret sharing. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2024(4):61–79, 2024.

[11] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 15718–15731, 2022.

[12] Wen jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. BumbleBee: Secure two-party inference framework for large transformers. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[13] Andes Y. L. Kei and Sherman S. M. Chow. SHAFT: Secure, handy, accurate, and fast transformer inference. In *Network and Distributed System Security Symposium (NDSS)*, 2025. https://www.ndss-symposium.org/ndss-paper/shaft-secure-handy-accurate-and-fast-transformer-inference/; Cryptology ePrint Archive, Paper 2025/2324, https://eprint.iacr.org/2025/2324.

[14] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1575–1590, 2020.

[15] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. CrypTen: Secure multi-party computation meets machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 4963–4975, 2021.

[16] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[17] Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P. Xing, and Hao Zhang. MPCFormer: Fast, performant and private transformer inference with MPC. In *International Conference on Learning Representations (ICLR)*, 2023.

[18] Zhengyi Li, Kang Yang, Jin Tan, Wen jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, and Jingwen Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. https://arxiv.org/abs/2411.15707.

[19] Junming Ma, Yancheng Zheng, Jun Feng, Derun Zhao, Haoqi Wu, Wenjing Fang, Jin Tan, Chaofan Yu, Benyu Zhang, and Lei Wang. SecretFlow-SPU: A performant and user-friendly framework for privacy-preserving machine learning. In *USENIX Annual Technical Conference (USENIX ATC)*, pages 17–33, 2023.

[20] Payman Mohassel and Yupeng Zhang. SecureML: A system for scalable privacy-preserving machine learning. In *IEEE Symposium on Security and Privacy (S&P)*, pages 19–38, 2017.

[21] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. BOLT: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE Symposium on Security and Privacy (S&P)*, pages 4753–4771, 2024.

[22] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 325–342, 2020.

[23] José Reis, Mehmet Ugurbil, Sameer Wagh, Ryan Henry, and Miguel de Vega. Wave hello to privacy: Efficient mixed-mode MPC using wavelet transforms. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2025(2):697–718, 2025. https://doi.org/10.56553/popets-2025-0083.

[24] Manuel B. Santos, Dimitris Mouris, Mehmet Ugurbil, Stanislaw Jarecki, José Reis, Shubho Sengupta, and Miguel de Vega. Curl: Private LLMs through wavelet-encoded look-up tables. In *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS)*, pages 1–31, 2024. Cryptology ePrint Archive, Paper 2024/1127, https://eprint.iacr.org/2024/1127.

[25] Kyle Storrier, Adithya Vadapalli, Allan Lyons, and Ryan Henry. Grotto: Screaming fast (2+1)-pc for $\mathbb{Z}_{2^n}$ via (2, 2)-dpfs. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2143–2157, 2023.

[26] Sijun Tan, Brian Knott, Yuan Tian, and David J. Wu. CryptGPU: Fast privacy-preserving machine learning on the GPU. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1021–1038, 2021.

[27] Mehmet Ugurbil, Dimitris Mouris, Manuel B. Santos, José Cabrero-Holgueras, Miguel de Vega, and Shubho Sengupta. Fission: Distributed privacy-preserving large language model inference. In *IEEE Conference on Communications and Network Security (CNS)*, pages 1–9, 2025. Cryptology ePrint Archive, Paper 2025/653, https://eprint.iacr.org/2025/653.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[29] Jean-Luc Watson, Sameer Wagh, and Raluca Ada Popa. Piranha: A GPU platform for secure computation. In *USENIX Security Symposium*, pages 827–844, 2022.